RAPPORT DE CONCEPTION · AWD 2026–2027

StayEasy

Réserver un séjour. Trouver une colocation.

Conception d'une plateforme en architecture microservices

Hébergements · Réservations · Maintenance · Colocation
Paiement · Notification · Utilisateurs

Spring Boot | RabbitMQ | Keycloak | Docker Compose

NOTRE ÉQUIPE

Molka Kammoun ● **Isra Khlifi** ● **Mohamed Khouni**
Samar Masmoudi ● **Nermine Karoui** ● **Lina Kazdaghli**

Encadrement : Linda Ourfelli · Oumayma Ibn El Fekih

Sommaire

Ce rapport présente la conception de StayEasy : le sujet, ses acteurs, le découpage en microservices, les données, les interactions et l'architecture globale.

01 Description du sujet 3
Contexte, problématique, solution et objectifs

02 Acteurs et fonctionnalités 5
Qui utilise StayEasy et pour quoi faire

03 Découpage en microservices 7
Sept services autonomes, leurs responsabilités et leurs règles clés

04 Données gérées par chaque service 12
Entités, méthodes, énumérations et couches de chaque service

05 Interactions entre les services 27
Interactions des sept services, événements RabbitMQ, scénarios clés

06 Diagramme d'architecture globale 41
Vue d'ensemble, composants et déploiement

07 Autres éléments de conception 43
API de chaque service, sécurité, risques, organisation, planning

08 Conclusion et perspectives 52
Bilan et prochaines étapes

3	7	7	30
acteurs	microservices	bases dédiées	figures

01 Description du sujet

1.1 Contexte

StayEasy est une plateforme de gestion des hébergements, des réservations et des services associés. Elle centralise dans un espace unique des opérations aujourd'hui dispersées : la recherche de logement, la réservation, le paiement, le suivi de la maintenance, la colocation et la communication entre les différents acteurs.

Le projet s'adresse à deux besoins complémentaires, résumés par la promesse de la plateforme : **réserver un séjour** (chambres et séjours) et **trouver une colocation** (annonces et partage).

1.2 Problématique

LA QUESTION CENTRALE

Comment aider les personnes en quête de repos ou d'une expérience unique à trouver un lieu calme proche de la nature, tout en donnant de la visibilité aux maisons d'hôtes isolées ?

La gestion des logements soulève plusieurs difficultés concrètes :

1. **Recherche et disponibilité**
   Trouver rapidement un hébergement adapté et connaître ses disponibilités réelles.
2. **Réservations et paiements**
   Gérer le cycle de vie d'une réservation et le paiement associé sans conflit de dates.
3. **Maintenance et colocation**
   Suivre l'état des chambres, les interventions et les annonces de colocation.
4. **Communication**
   Faire circuler l'information entre voyageurs, propriétaires et équipe technique.

1.3 Solution proposée

StayEasy répond à ces difficultés par une plateforme unique, construite sur **sept services autonomes**, chacun avec sa base de données dédiée et des contrats d'API communs. Ce choix isole les responsabilités, permet à chaque membre de l'équipe de développer son service en parallèle et rend le système plus évolutif.

Le défi d'architecture : les données et les actions à coordonner (chambres, dates, paiements, interventions) sont réparties entre plusieurs services. La conception doit donc garantir un parcours cohérent d'un service à l'autre.

1.4 Objectifs et périmètre

- Permettre de rechercher, consulter et réserver un hébergement, avec un paiement simulé.
- Offrir aux propriétaires des outils de gestion des hébergements, des chambres, des réservations et des annonces.
- Proposer un parcours de colocation : séjours partagés, matching de compatibilité, activités et votes de groupe.

- Suivre les interventions techniques et l'état opérationnel des chambres.
- Informer les utilisateurs par des notifications sans bloquer les opérations principales.
- Sécuriser l'accès par rôles grâce à une authentification centralisée.

02 Acteurs et fonctionnalités

2.1 Les acteurs

Trois types d'acteurs interagissent avec StayEasy. Leurs droits sont portés par des rôles gérés dans Keycloak.

V

Voyageur / Étudiant

Trouve une chambre, réserve et suit son séjour. Peut aussi rejoindre une colocation.

P

Propriétaire

Gère les chambres, les hébergements, les réservations et les annonces.

T

Équipe technique

Suit les interventions et l'état opérationnel des chambres.

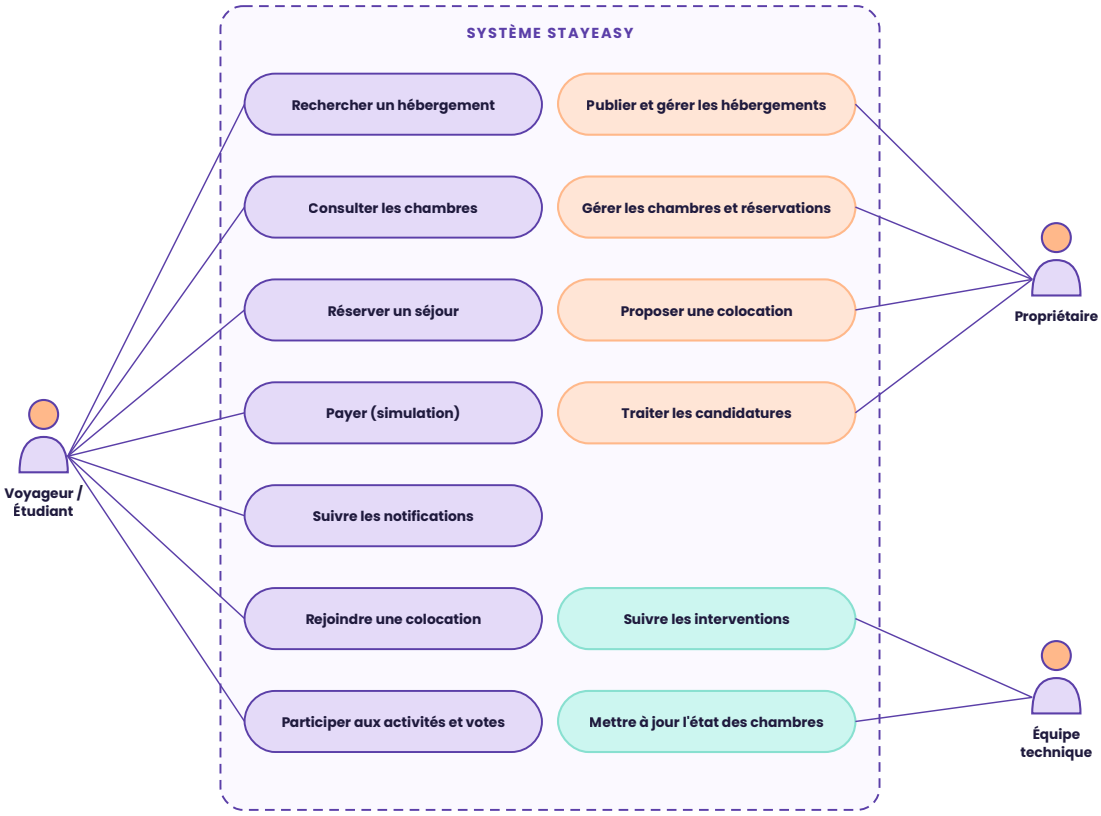


Figure 1 · Diagramme des cas d'utilisation de StayEasy

2.2 Fonctionnalités principales

Les fonctionnalités s'organisent en **deux parcours complémentaires**.

Réservation de séjour

- Rechercher un hébergement
- Consulter les chambres
- Réserver et payer en simulation
- Suivre les notifications

Gestion et colocation

- Publier et gérer les logements
- Proposer une colocation
- Traiter les candidatures
- Suivre les interventions

2.3 Détail par acteur

Acteur	Fonctionnalités	Services concernés
Voyageur / Étudiant	Recherche d'hébergements, consultation des chambres, réservation, paiement simulé, suivi du séjour, notifications, demande pour rejoindre un séjour partagé, participation aux activités, votes et retours d'expérience.	Accommodation, Reservation, Payment, Colocation, Notification, User
Propriétaire	Publication et gestion des hébergements et chambres, gestion des réservations, création d'annonces de colocation, traitement des candidatures.	Accommodation, Reservation, Colocation, Notification, User
Équipe technique	Suivi des interventions, mise à jour de l'état opérationnel des chambres.	Maintenance, Accommodation, User

03 Découpage en microservices

Chaque service possède un périmètre métier précis, sa base de données et son port.

3.1 Principes de découpage

Autonomie
Un service = un domaine métier, développé, testé et déployé indépendamment.

Base par service
Aucune base partagée : chaque service est seul maître de ses données.

Contrats communs
Les services dialoguent uniquement par API REST ou événements documentés.

3.2 Responsabilités des services

Réf.	Service	Responsabilité	Base	Port
MS1	**Accommodation**	Hébergements, chambres et équipements	H2	8081
MS2	**Reservation**	Réservations et calendrier	H2	8082
MS3	**Maintenance**	Interventions techniques	H2	8083
MS4	**Colocation**	Séjours partagés, annonces et candidatures	MySQL	8084
MS5	**Payment**	Paiements simulés et remboursements	H2	8085
MS6	**Notification**	Messages et historique	MySQL	8086
MS7	**User (commun)**	Profils métier	PostgreSQL	8087

3.3 Fiches des services

Chacun des sept services est présenté selon la même grille : mission, quatre capacités principales et règle métier clé.

MS1 · port 8081

Accommodation H2 :8081

Il porte le catalogue de la plateforme : hébergements (maisons d'hôtes), chambres et équipements. Il est la source de vérité sur la capacité, le prix par nuit et l'état opérationnel des chambres.

1

Publication d'hébergements

Créer, modifier, publier ou suspendre un hébergement par son propriétaire.

2

Gestion des chambres

Ajouter des chambres, fixer capacité et prix, suivre leur statut.

3

Recherche et consultation

Filtrer par ville, type, capacité, prix et équipements.

4

État opérationnel

Exposer la disponibilité réelle à Reservation et Colocation ; recevoir les changements d'état de Maintenance.

Règle clé · Seul le propriétaire d'un hébergement peut le modifier ; une chambre EN_MAINTENANCE ou HORS_SERVICE n'est jamais proposée à la réservation.

MS2 · port 8082

Reservation H2 :8082

Il gère le cycle de vie des réservations : de la création à la confirmation, en passant par la modification et l'annulation. Il vérifie la disponibilité auprès d'Accommodation, déclenche le paiement auprès de Payment, puis publie des événements consommés par Notification.

1

Création et blocage

Créer une réservation EN_ATTENTE et bloquer les nuits de façon atomique.

2

Paiement coordonné

Demander le paiement à Payment, puis confirmer ou annuler selon le résultat.

3

Modification et annulation

Modifier les dates, annuler, libérer les nuits et déclencher le remboursement.

4

Calendrier et réconciliation

Exposer les périodes occupées et vérifier les réservations restées en attente après un timeout.

Règle clé · Pour éviter la double réservation, les nuits sont bloquées de façon atomique au moment de la création de la réservation.

MS3 · port 8083

Maintenance H2 :8083

Il suit les interventions techniques sur les chambres : signalement, priorité, planification, clôture et commentaires de l'équipe. Il garde le catalogue à jour en changeant l'état opérationnel des chambres.

1

Signalement

Déclarer un problème sur une chambre avec son type et sa priorité.

2

Planification et affectation

Fixer une date d'intervention et assigner un technicien.

3

Suivi et commentaires

Faire avancer le statut et échanger via les commentaires de l'équipe.

4

Synchronisation du catalogue

Passer la chambre EN_MAINTENANCE, puis la remettre DISPONIBLE à la clôture.

Règle clé · Une chambre ne peut pas être mise en maintenance sur une période couverte par une réservation confirmée (contrôle auprès de Reservation).

MS4 · port 8084

Colocation MySQL :8084

Il propose des hébergements partagés et des expériences : créer des séjours partagés, rapprocher les participants compatibles et organiser des activités ensemble.

1

Gestion de groupe

Créer ou rejoindre un séjour partagé, gérer les participants et leurs demandes.

2

Matching intelligent

Analyser les préférences et recommander les groupes les plus compatibles grâce à un score.

3

Planification d'activités

Créer des activités de groupe, organiser le calendrier et gérer les inscriptions.

4

Communauté et interaction

Échanger, voter ensemble et partager ses retours sur l'expérience vécue.

Règle clé · Un séjour ne peut pas dépasser maxParticipants ; l'adhésion passe par une candidature (PENDING) traitée par l'organisateur.

MS5 · port 8085

Payment H2 :8085

Il traite des paiements simulés et les remboursements associés. Il garantit qu'une même demande n'est jamais traitée deux fois et publie le résultat de chaque opération.

1

Initiation du paiement

Créer le paiement et simuler l'autorisation (accepté ou refusé).

2

Idempotence

Reconnaître une demande déjà traitée grâce à la clé idempotencyKey.

3

Remboursements

Rembourser totalement ou partiellement, toujours rattaché au paiement d'origine.

4

Paiement collectif

Traiter les demandes de groupe émises par Colocation et publier le résultat.

Règle clé · Deux demandes avec la même clé d'idempotence ne produisent qu'un seul paiement ; la somme des remboursements ne dépasse jamais le montant payé.

MS6 · port 8086

Notification MySQL :8086

Il informe les utilisateurs sans jamais bloquer les opérations principales : il consomme les événements métier, prépare les messages à partir de modèles et conserve l'historique.

1

Consommation d'événements

Écouter les événements publiés dans RabbitMQ par les services métier.

2

Modèles de message

Générer le texte à partir de gabarits identifiés par un code et des variables.

3

Canaux multiples

Envoyer par e-mail, SMS (simulé), notification push ou message dans l'application.

4

Historique et suivi

Consulter, marquer comme lu et rejouer les envois en échec.

Règle clé · Un échec d'envoi ne remonte jamais vers le service émetteur ; chaque événement est dédupliqué grâce à son identifiant de message.

MS7 · port 8087

User PostgreSQL :8087

Il stocke les profils métier des utilisateurs ; l'identité elle-même (mot de passe, connexion) reste gérée par Keycloak, avec lequel il est lié par keycloakId.

1

Inscription et profil

Créer le profil métier après la première connexion Keycloak.

2

Profils spécialisés

ProfilVoyageur (université, préférences, budget) et ProfilProprietaire (société, adresse).

3

Rôles et statut

Refléter les rôles Keycloak et activer ou suspendre un compte.

4

Annuaire interne

Répondre aux autres services : vérifier un participant, un propriétaire, un technicien ou des coordonnées.

Règle clé · Aucun mot de passe n'est stocké par StayEasy : l'identité reste dans Keycloak et seul le keycloakId fait le lien.

3.4 Services transverses

Composant	Rôle dans la plateforme
API Gateway	Spring Cloud Gateway : point d'entrée unique, routage des requêtes vers les services.
Eureka	Découverte de services : les microservices s'enregistrent et se retrouvent sans adresse fixe.
Keycloak	Connexion et rôles via OIDC ; la Gateway contrôle les jetons.
RabbitMQ	Bus d'événements métier pour la communication asynchrone.
Docker Compose	Orchestration locale de l'ensemble des conteneurs.

04 Données gérées par chaque service

Chaque service gère ses propres entités dans une base dédiée ; les références entre services passent par des identifiants. Les sept services sont documentés de la même façon : **diagramme de classes** (attributs, méthodes, relations), **énumérations** et **vue en couches** (contrôleur, service, dépôt, intégration).

Notation. « – » attribut privé, « + » méthode publique. Les attributs typés par une énumération apparaissent en **rose** et sont reliés à leur énumération par un trait pointillé. ◆ indique une composition. Chaque service possède ses propres énumérations : entre services, les valeurs circulent sous forme de texte dans les échanges JSON.

À lire avant. Les modèles reprennent ceux de la présentation (Reservation, Colocation) et les complètent par des propositions cohérentes avec les responsabilités de chaque service. Éléments ajoutés : entité NuitBloquee (Reservation), motifAnnulation, EvenementTraite (Notification), statut de Remboursement, identifiants des profils (User) et multiplicités de Colocation précisées (0..*). Ils seront affinés lors du développement.

Service	Base	Entités principales	Énumérations
Accommodation	H2	Hebergement, Chambre, Equipement	TypeHebergement, StatutHebergement, TypeChambre, StatutChambre, CategorieEquipement
Reservation	H2	Reservation, NuitBloquee	StatutReservation, MotifAnnulation
Maintenance	H2	Intervention, CommentaireIntervention	TypeIntervention, PrioriteIntervention, StatutIntervention
Colocation	MySQL	SharedStay, StayMember, Activity, GroupVote, ExperienceFeedback, ActivityParticipation, VoteResponse	ExperienceType, StayStatus, MemberRole, MembershipStatus, ParticipationStatus, VoteStatus
Payment	H2	Paiement, Remboursement	MethodePaiement, StatutPaiement, MotifRemboursement, StatutRemboursement
Notification	MySQL	Notification, ModeleMessage, EvenementTraite	CanalNotification, TypeNotification, StatutNotification
User	PostgreSQL	Utilisateur, ProfilVoyageur, ProfilProprietaire	RoleUtilisateur, StatutCompte, TypeExperience

4.1 Service Accommodation (MS1)

Il porte le catalogue : un hébergement (maison d'hôtes) possède des chambres et propose des équipements. La capacité et le prix par nuit sont consultés par Reservation et Colocation.

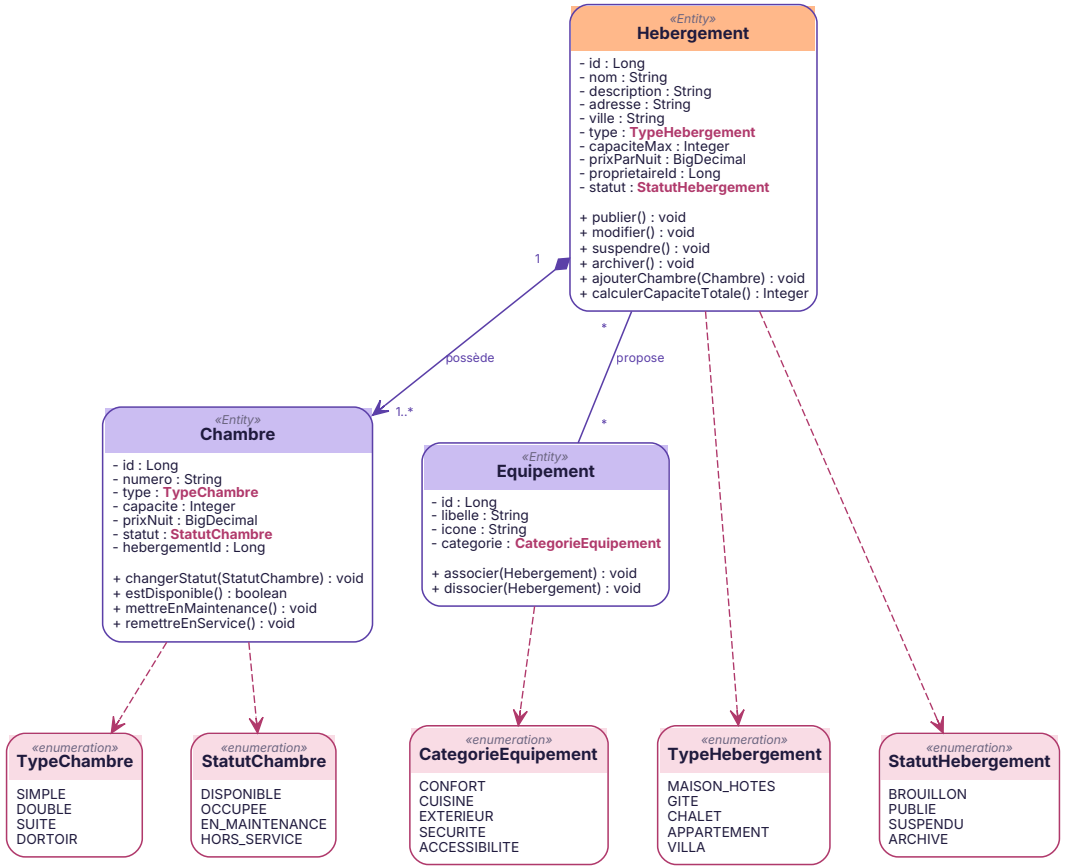


Figure 2 · Diagramme de classes du service Accommodation (attributs, méthodes, relations et énumérations)

Accommodation · énumérations

Énumération	Valeurs	Utilisée par
TypeHebergement	`MAISON_HOTES` · `GITE` · `CHALET` · `APPARTEMENT` · `VILLA`	`Hebergement.type`
StatutHebergement	`BROUILLON` · `PUBLIE` · `SUSPENDU` · `ARCHIVE`	`Hebergement.statut`
TypeChambre	`SIMPLE` · `DOUBLE` · `SUITE` · `DORTOIR`	`Chambre.type`
StatutChambre	`DISPONIBLE` · `OCCUPEE` · `EN_MAINTENANCE` · `HORS_SERVICE`	`Chambre.statut`
CategorieEquipement	`CONFORT` · `CUISINE` · `EXTERIEUR` · `SECURITE` · `ACCESSIBILITE`	`Equipement.categorie`

Accommodation · architecture en couches

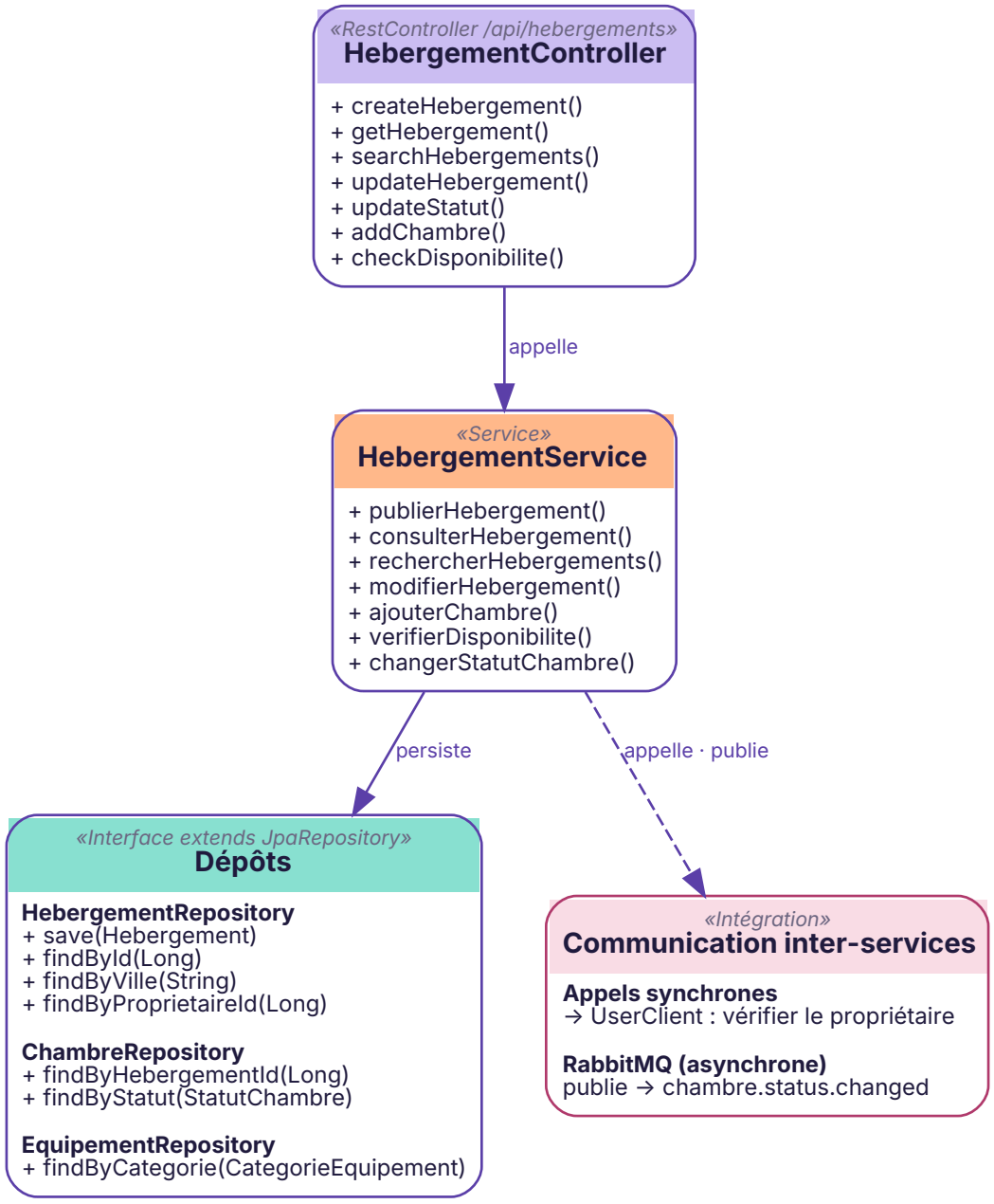


Figure 3 · Architecture en couches du service Accommodation (contrôleurs, services, dépôts, intégration)

4.2 Service Reservation (MS2)

Architecture en couches classique de Spring Boot : un contrôleur REST appelle un service métier, qui persiste via des dépôts JPA. L'entité Reservation référence l'utilisateur et l'hébergement par identifiants (userId, hebergementId), jamais par jointure inter-bases.

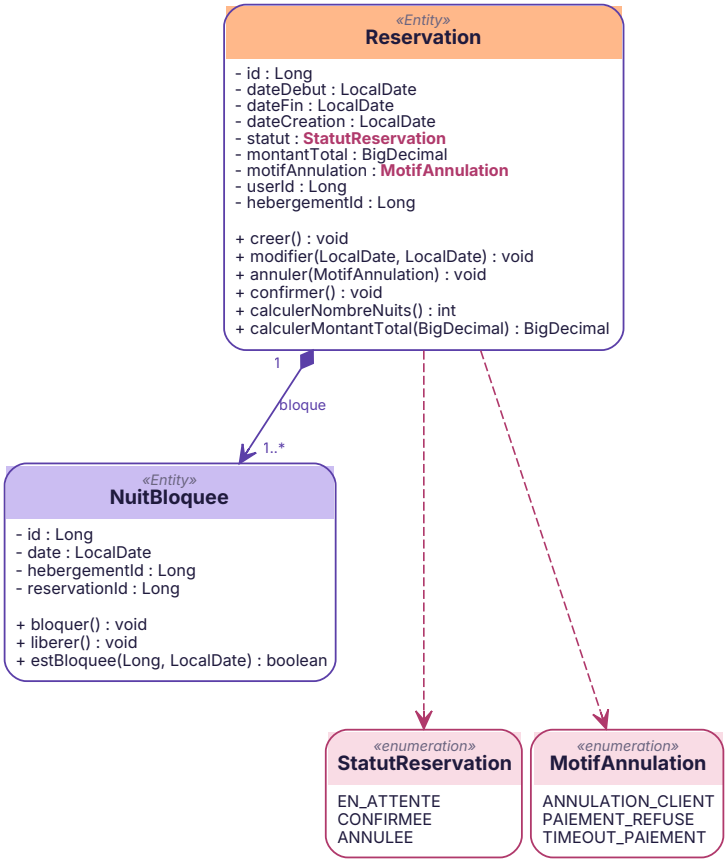


Figure 4 · Diagramme de classes du service Reservation (attributs, méthodes, relations et énumérations)

Reservation · énumérations

Énumération	Valeurs	Utilisée par
StatutReservation	`EN_ATTENTE` · `CONFIRMEE` · `ANNULEE`	`Reservation.statut`
MotifAnnulation	`ANNULATION_CLIENT` · `PAIEMENT_REFUSE` · `TIMEOUT_PAIEMENT`	`Reservation.motifAnnulation`

Reservation · architecture en couches

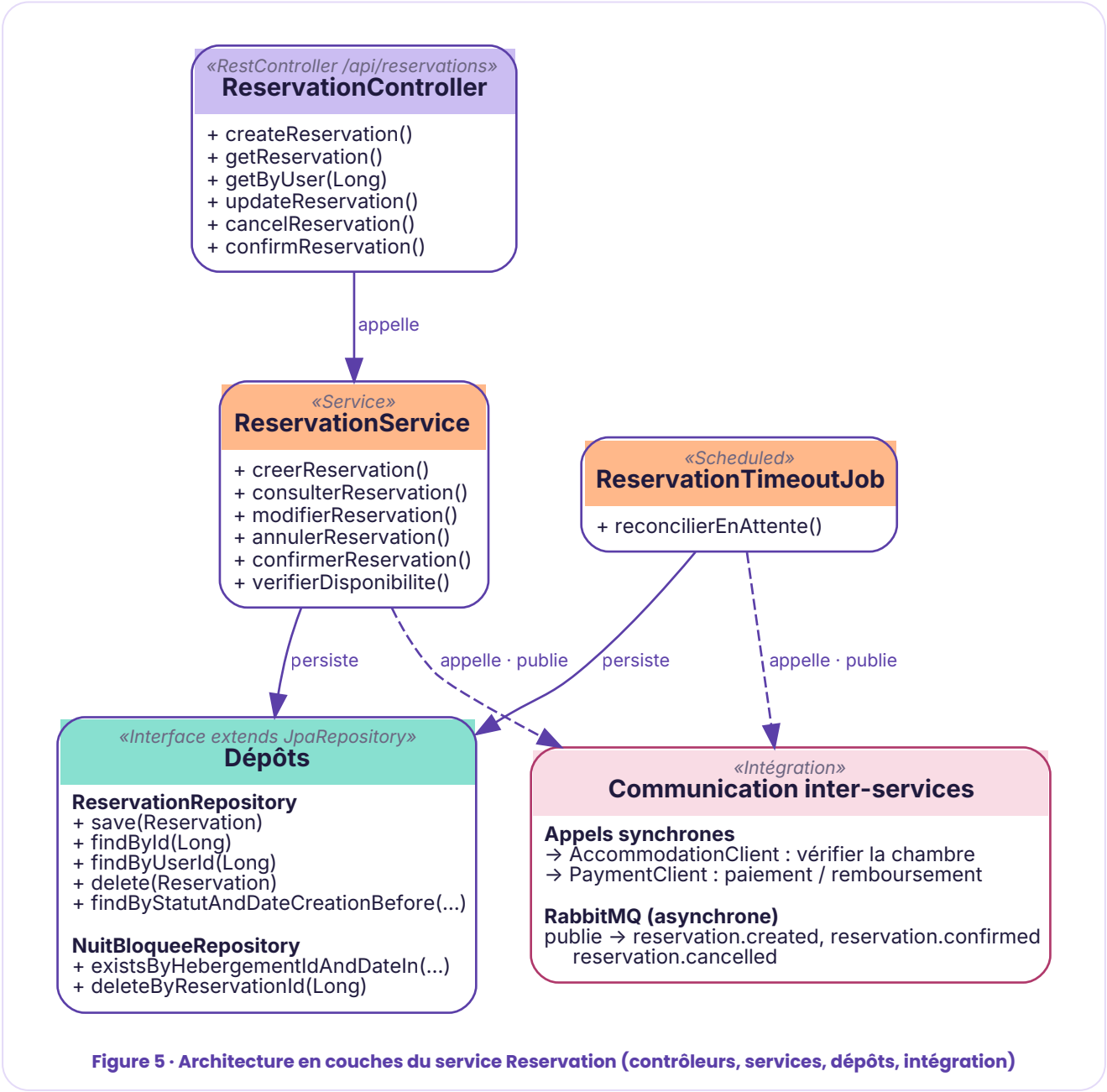


Figure 5 · Architecture en couches du service Reservation (contrôleurs, services, dépôts, intégration)

4.3 Service Maintenance (MS3)

Il suit les interventions techniques sur les chambres : signalement, priorité, planification, clôture et commentaires de l'équipe.

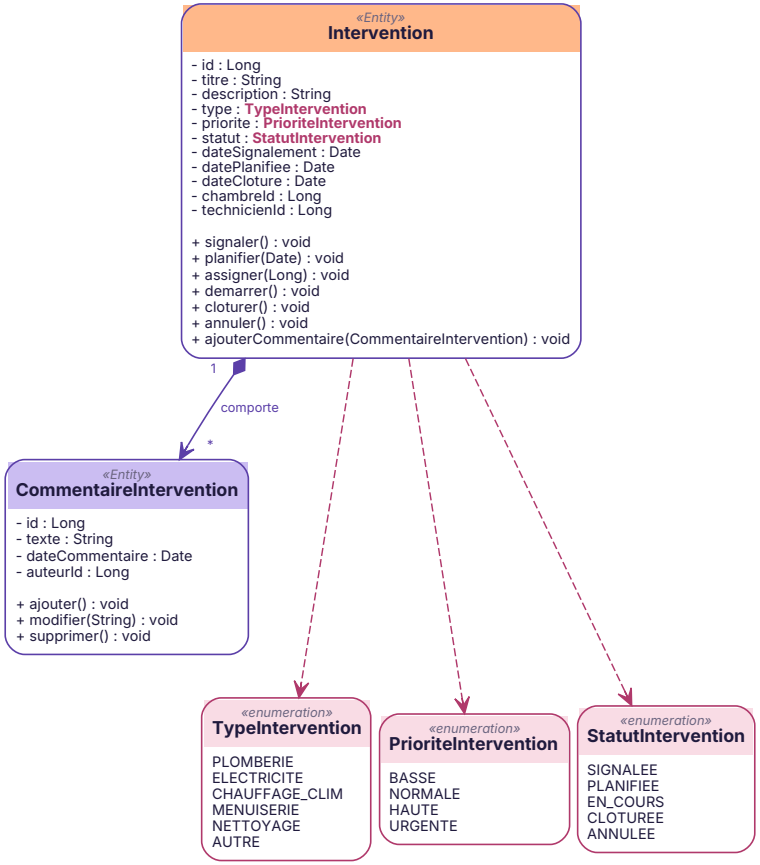


Figure 6 · Diagramme de classes du service Maintenance (attributs, méthodes, relations et énumérations)

Maintenance · énumérations

Énumération	Valeurs	Utilisée par
TypeIntervention	`PLOMBERIE` · `ELECTRICITE` · `CHAUFFAGE_CLIM` · `MENUISERIE` · `NETTOYAGE` · `AUTRE`	`Intervention.type`
PrioriteIntervention	`BASSE` · `NORMALE` · `HAUTE` · `URGENTE`	`Intervention.priorite`
StatutIntervention	`SIGNALEE` · `PLANIFIEE` · `EN_COURS` · `CLOTUREE` · `ANNULEE`	`Intervention.statut`

Maintenance · architecture en couches

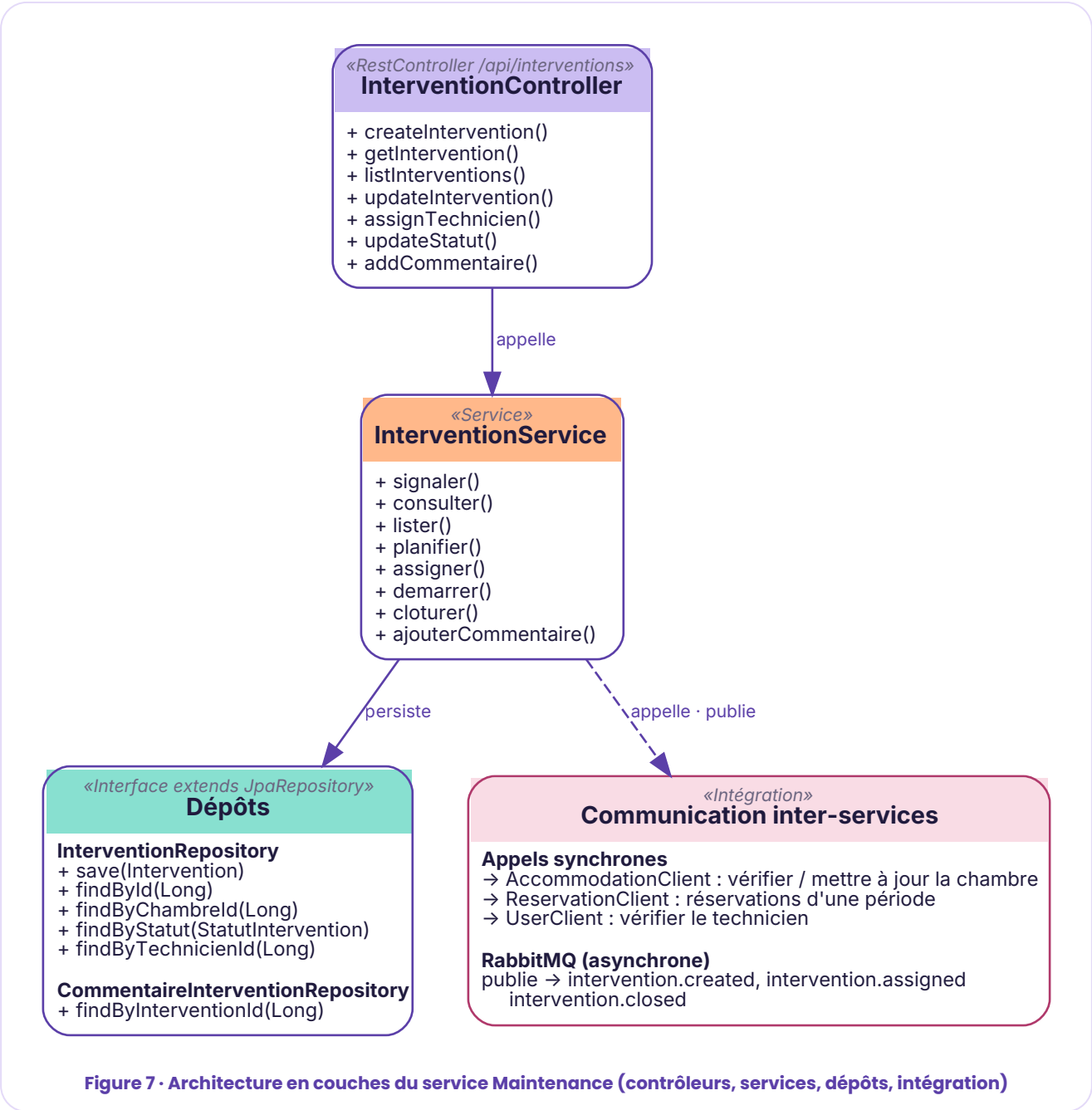


Figure 7 · Architecture en couches du service Maintenance (contrôleurs, services, dépôts, intégration)

4.4 Service Colocation (MS4)

Le séjour partagé (SharedStay) est l'agrégat central : il regroupe les membres, les activités, les votes de groupe et les retours d'expérience. Les participations aux activités et les réponses aux votes en dépendent.

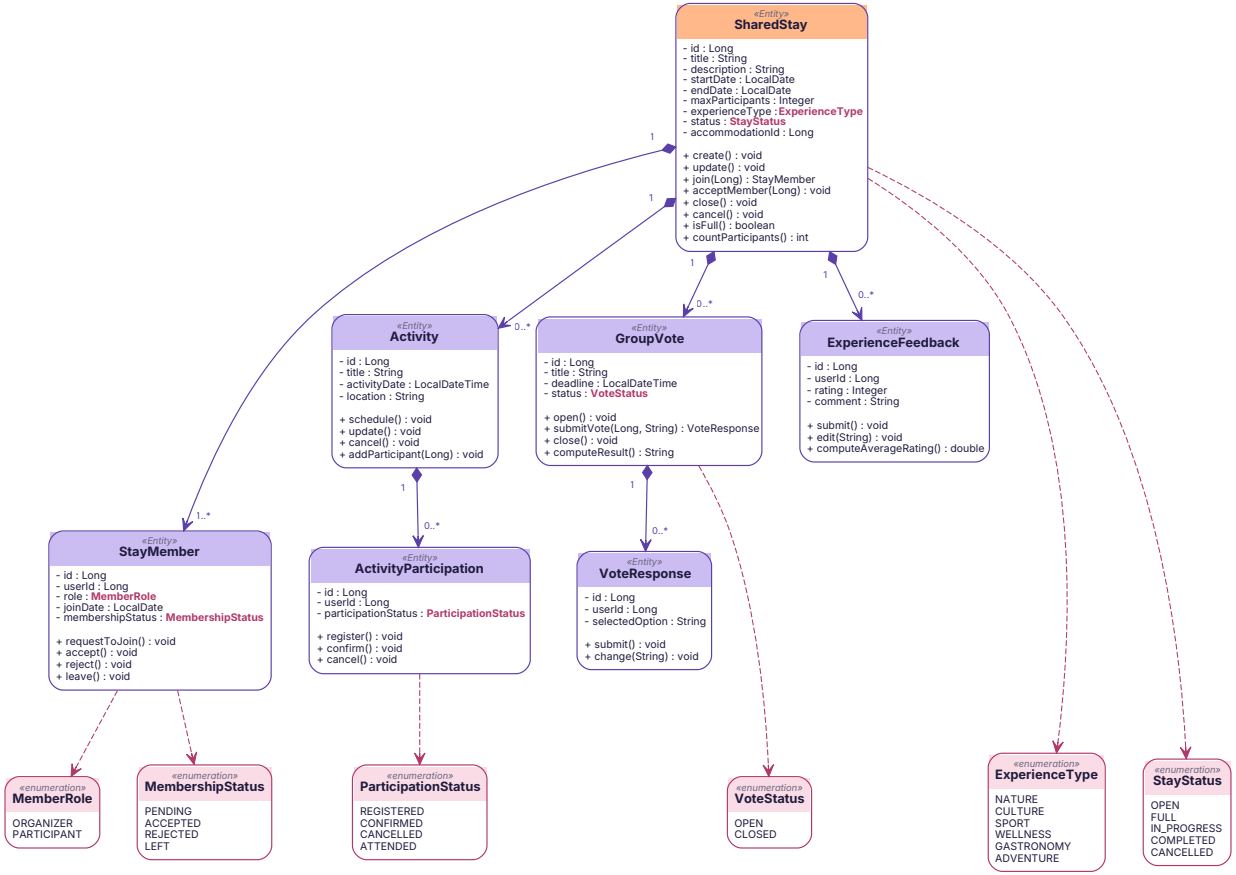


Figure 8 · Diagramme de classes du service Colocation (attributs, méthodes, relations et énumérations)

Colocation · énumérations

Énumération	Valeurs	Utilisée par
ExperienceType	NATURE · CULTURE · SPORT · WELLNESS · GASTRONOMY · ADVENTURE	`SharedStay.experienceType`
StayStatus	OPEN · FULL · IN_PROGRESS · COMPLETED · CANCELLED	`SharedStay.status`
MemberRole	ORGANIZER · PARTICIPANT	`StayMember.role`
MembershipStatus	PENDING · ACCEPTED · REJECTED · LEFT	`StayMember.membershipStatus`
ParticipationStatus	REGISTERED · CONFIRMED · CANCELLED · ATTENDED	`ActivityParticipation.participationStatus`
VoteStatus	OPEN · CLOSED	`GroupVote.status`

Colocation · architecture en couches

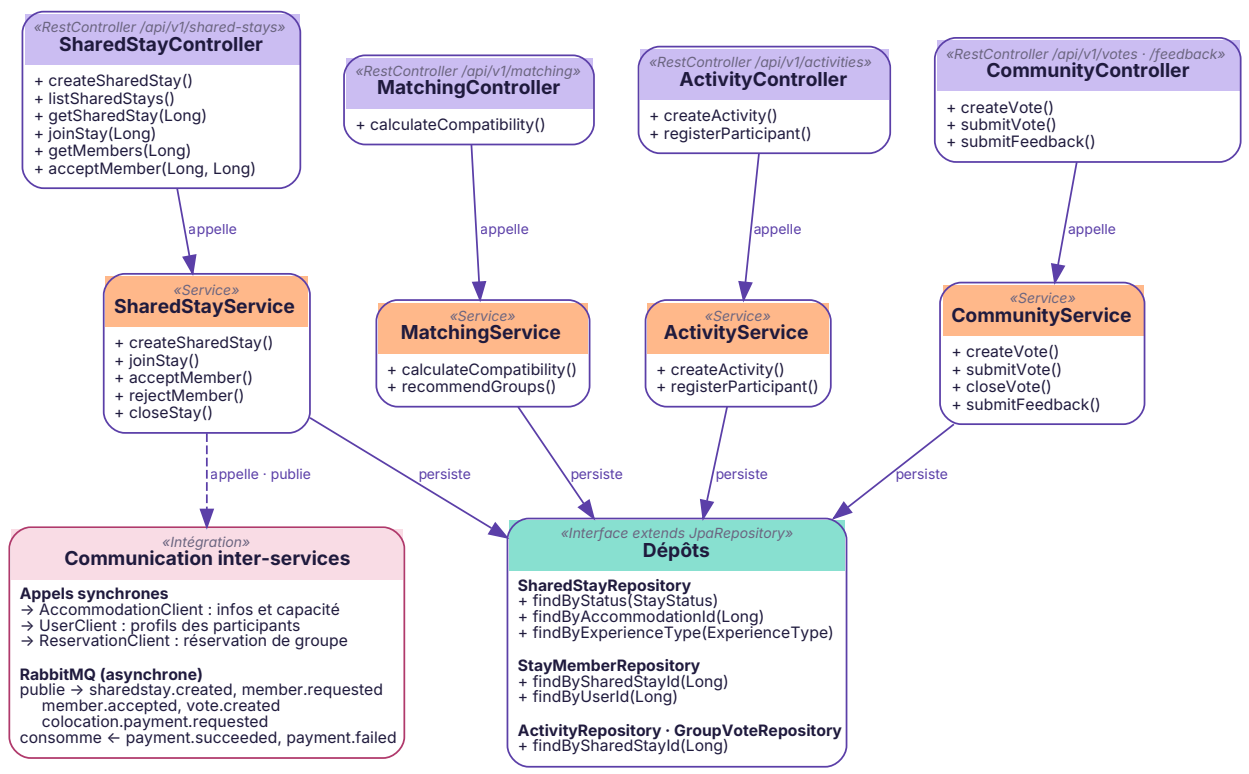


Figure 9 · Architecture en couches du service Colocation (contrôleurs, services, dépôts, intégration)

4.5 Service Payment (MS5)

Les paiements sont simulés. La clé d'idempotence évite de traiter deux fois la même demande, et les remboursements se rattachent au paiement d'origine.

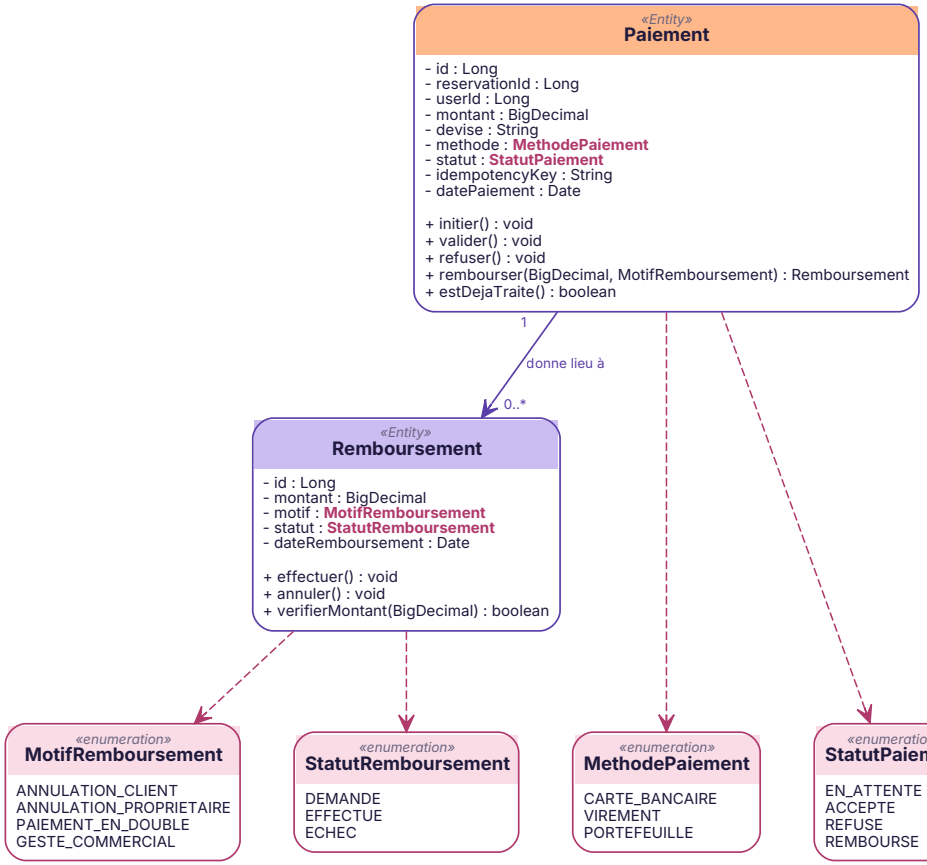


Figure 10 · Diagramme de classes du service Payment (attributs, méthodes, relations et énumérations)

Payment · énumérations

Énumération	Valeurs	Utilisée par
MethodePaiement	`CARTE_BANCAIRE` · `VIREMENT` · `PORTEFEUILLE`	`Paiement.methode`
StatutPaiement	`EN_ATTENTE` · `ACCEPTE` · `REFUSE` · `REMBOURSE`	`Paiement.statut`
MotifRemboursement	`ANNULATION_CLIENT` · `ANNULATION_PROPRIETAIRE` · `PAIEMENT_EN_DOUBLE` · `GESTE_COMMERCIAL`	`Remboursement.motif`
StatutRemboursement	`DEMANDE` · `EFFECTUE` · `ECHEC`	`Remboursement.statut`

Payment · architecture en couches

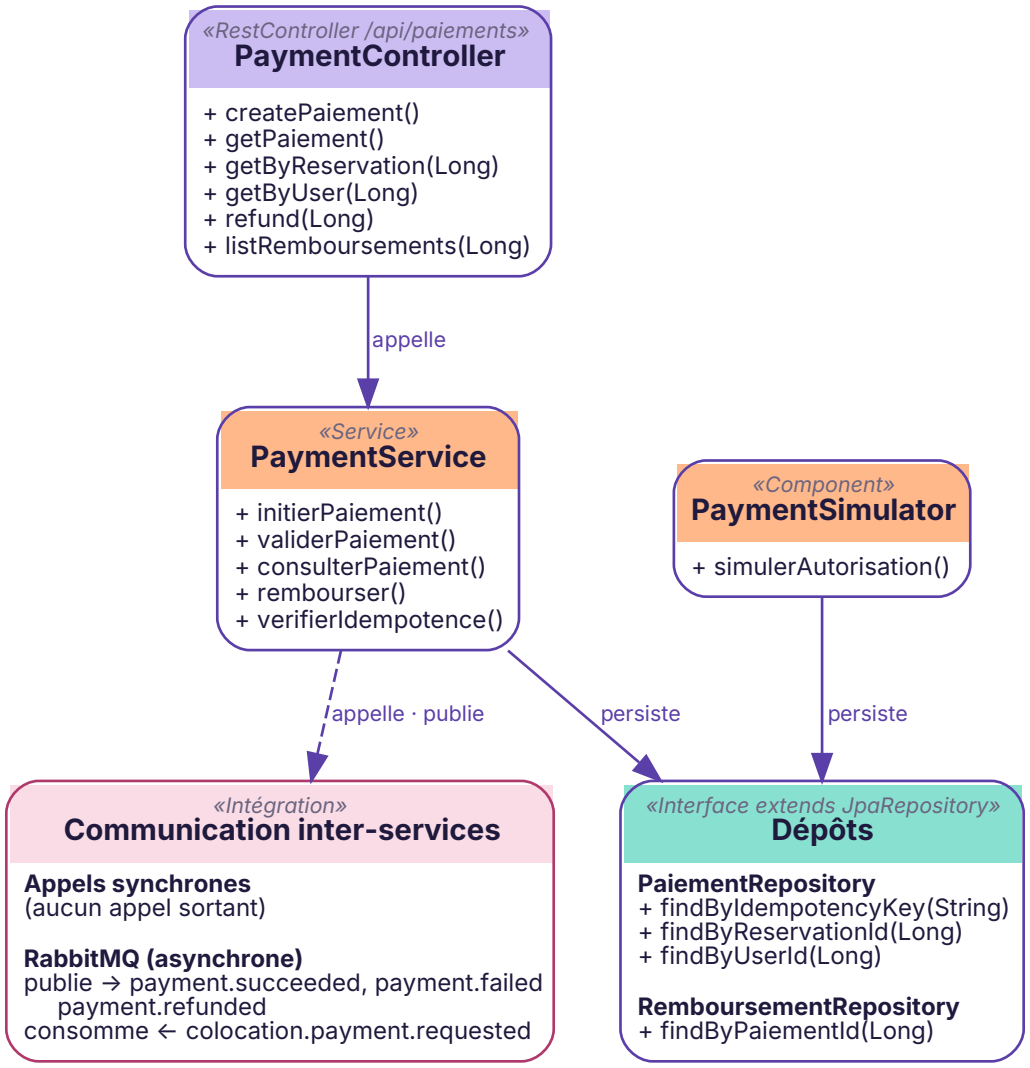


Figure 11 · Architecture en couches du service Payment (contrôleurs, services, dépôts, intégration)

4.6 Service Notification (MS6)

Le service Notification conserve l'historique des messages envoyés. Il consomme les événements des autres services et déduplique les messages reçus pour qu'un événement ne produise qu'une seule notification.

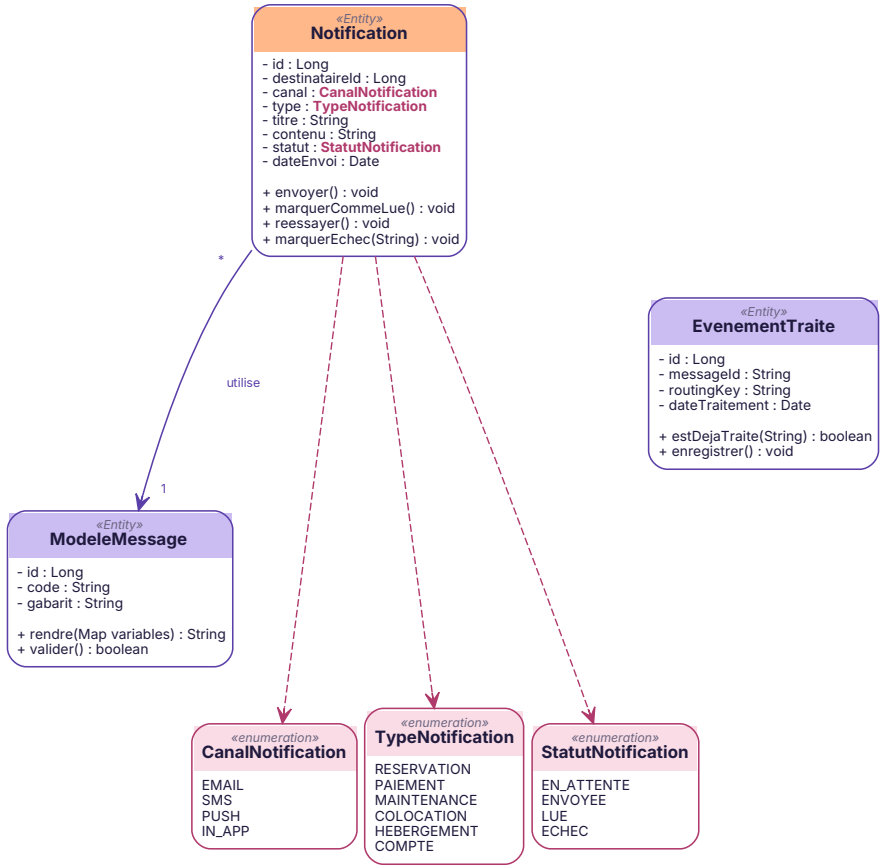


Figure 12 · Diagramme de classes du service Notification (attributs, méthodes, relations et énumérations)

Notification · énumérations

Énumération	Valeurs	Utilisée par
CanalNotification	`EMAIL` · `SMS` · `PUSH` · `IN_APP`	`Notification.canal`
TypeNotification	`RESERVATION` · `PAIEMENT` · `MAINTENANCE` · `COLOCATION` · `HEBERGEMENT` · `COMPTE`	`Notification.type`
StatutNotification	`EN_ATTENTE` · `ENVOYEE` · `LUE` · `ECHEC`	`Notification.statut`

Notification · architecture en couches

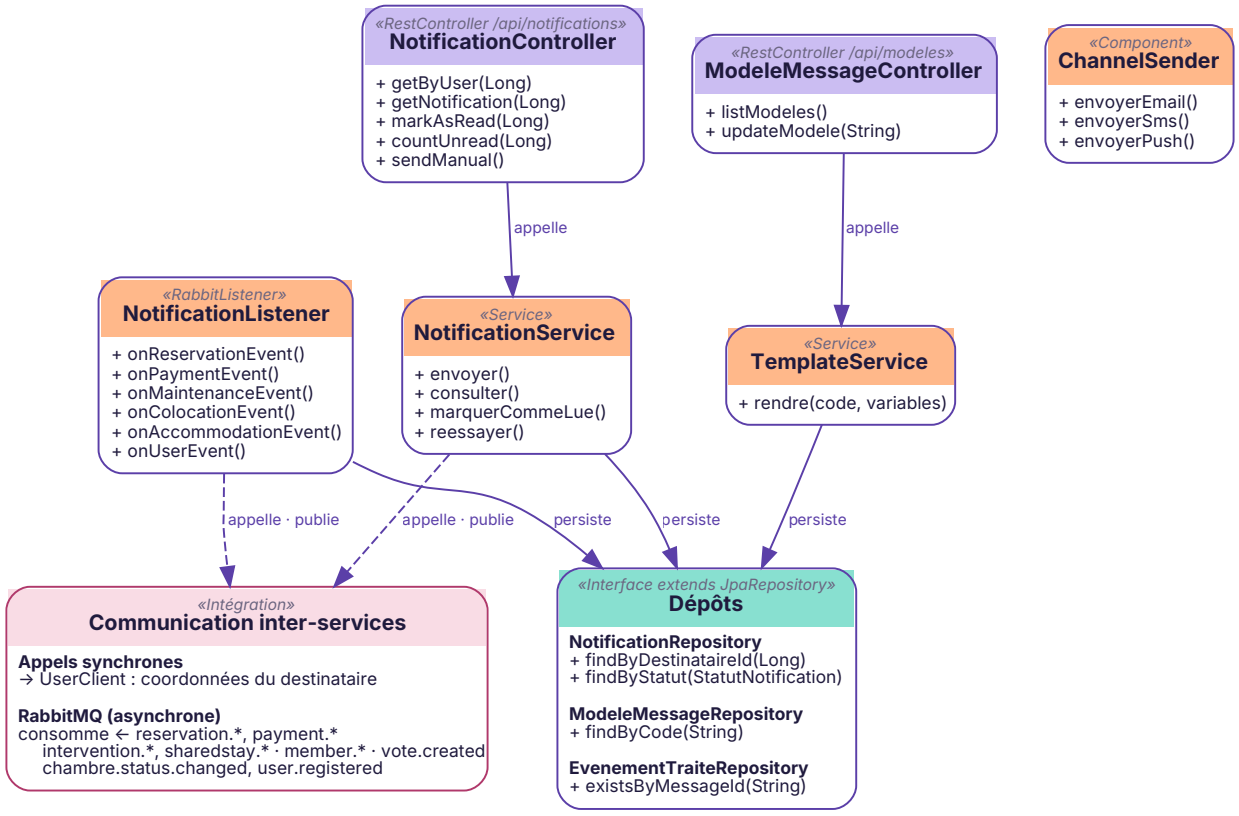


Figure 13 · Architecture en couches du service Notification (contrôleurs, services, dépôts, intégration)

4.7 Service User (MS7)

Le service User stocke les profils métier ; l'identité elle-même est gérée par Keycloak (lien via keycloakId). Il est commun à toute la plateforme.

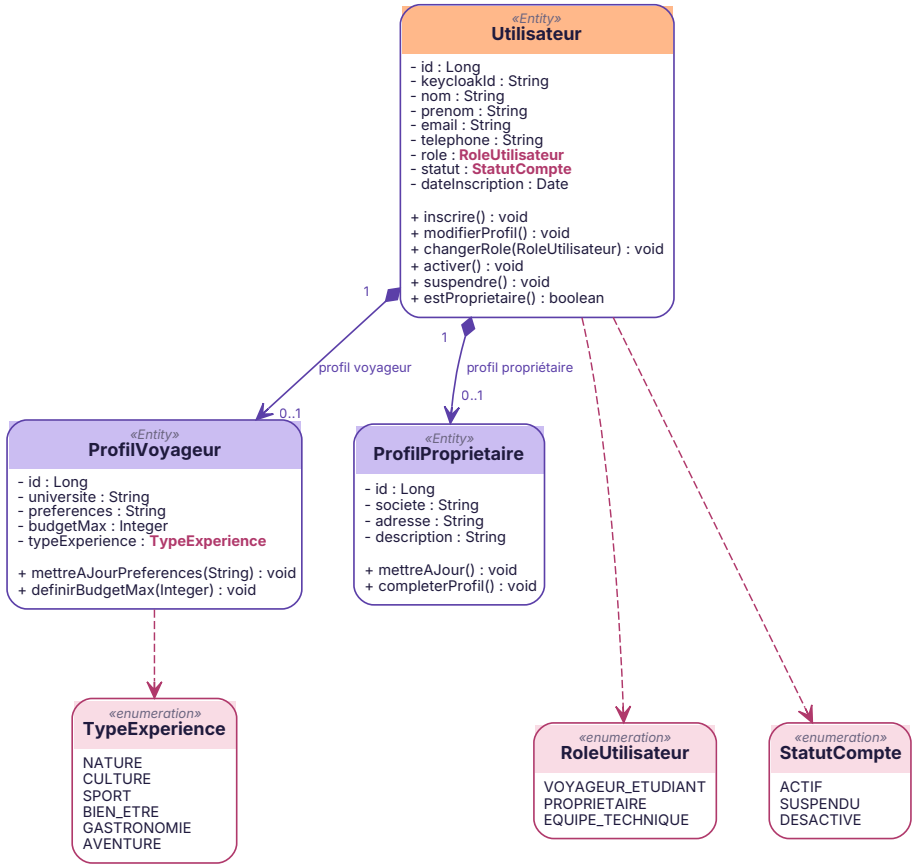


Figure 14 · Diagramme de classes du service User (attributs, méthodes, relations et énumérations)

User · énumérations

Énumération	Valeurs	Utilisée par
RoleUtilisateur	`VOYAGEUR_ETUDIANT` · `PROPRIETAIRE` · `EQUIPE_TECHNIQUE`	`Utilisateur.role`
StatutCompte	`ACTIF` · `SUSPENDU` · `DESACTIVE`	`Utilisateur.statut`
TypeExperience	`NATURE` · `CULTURE` · `SPORT` · `BIEN_ETRE` · `GASTRONOMIE` · `AVENTURE`	`ProfilVoyageur.typeExperience`

User · architecture en couches

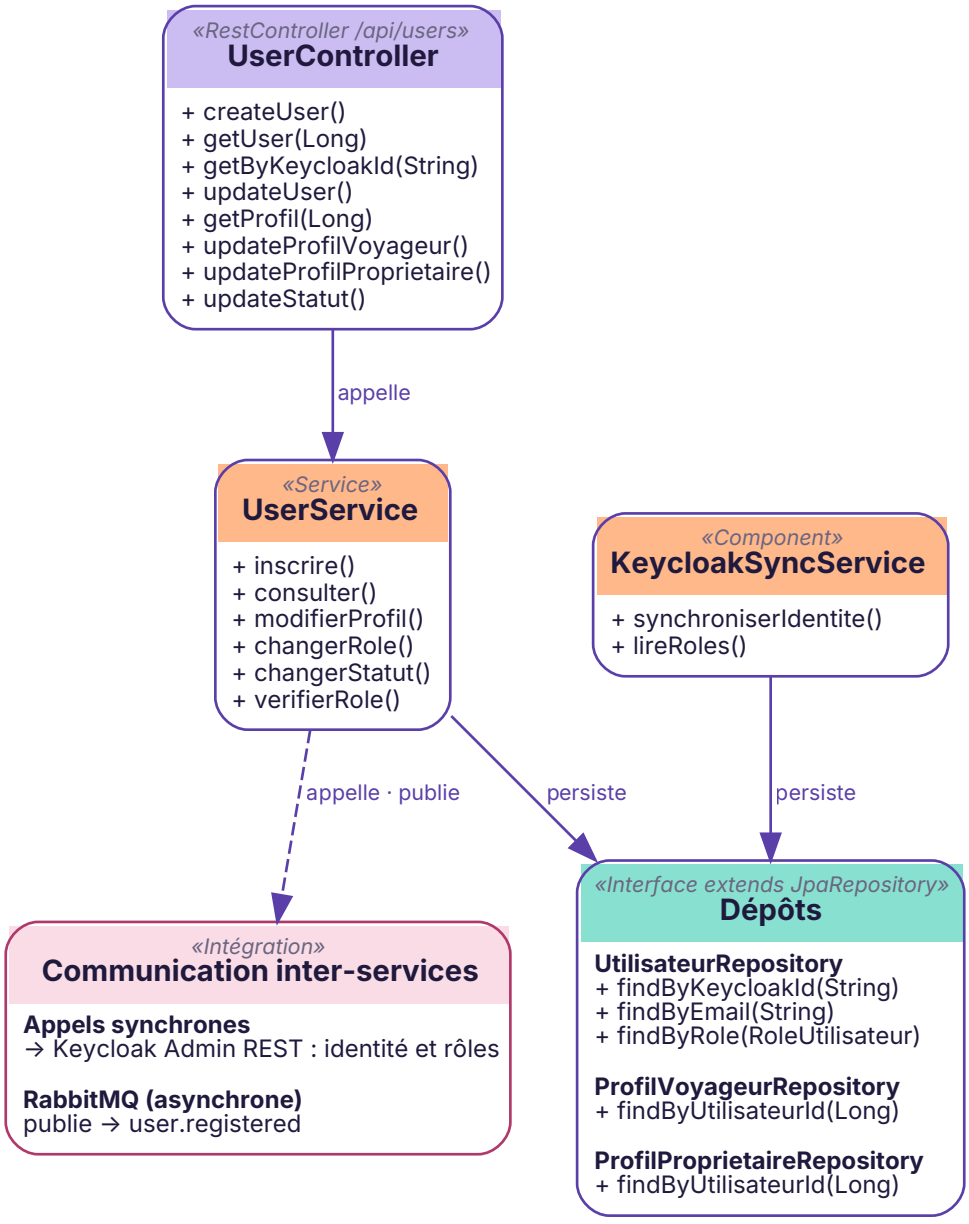


Figure 15 · Architecture en couches du service User (contrôleurs, services, dépôts, intégration)

05 Interactions entre les services

Deux modes de communication : synchrone (REST) quand une réponse immédiate est nécessaire, asynchrone (événements) pour informer sans bloquer. Cette partie décrit les interactions des **sept** services.

5.1 Synchrone ou asynchrone

Synchrone · REST / HTTP
Échange de données au format JSON, avec appels OpenFeign entre services. Utilisé pour vérifier un hébergement, vérifier des participants ou coordonner une réservation.

Asynchrone · événements
Les services publient des événements métier dans RabbitMQ. Notification les consomme : l'envoi du message ne bloque jamais la réservation.

5.2 Vue d'ensemble des interactions

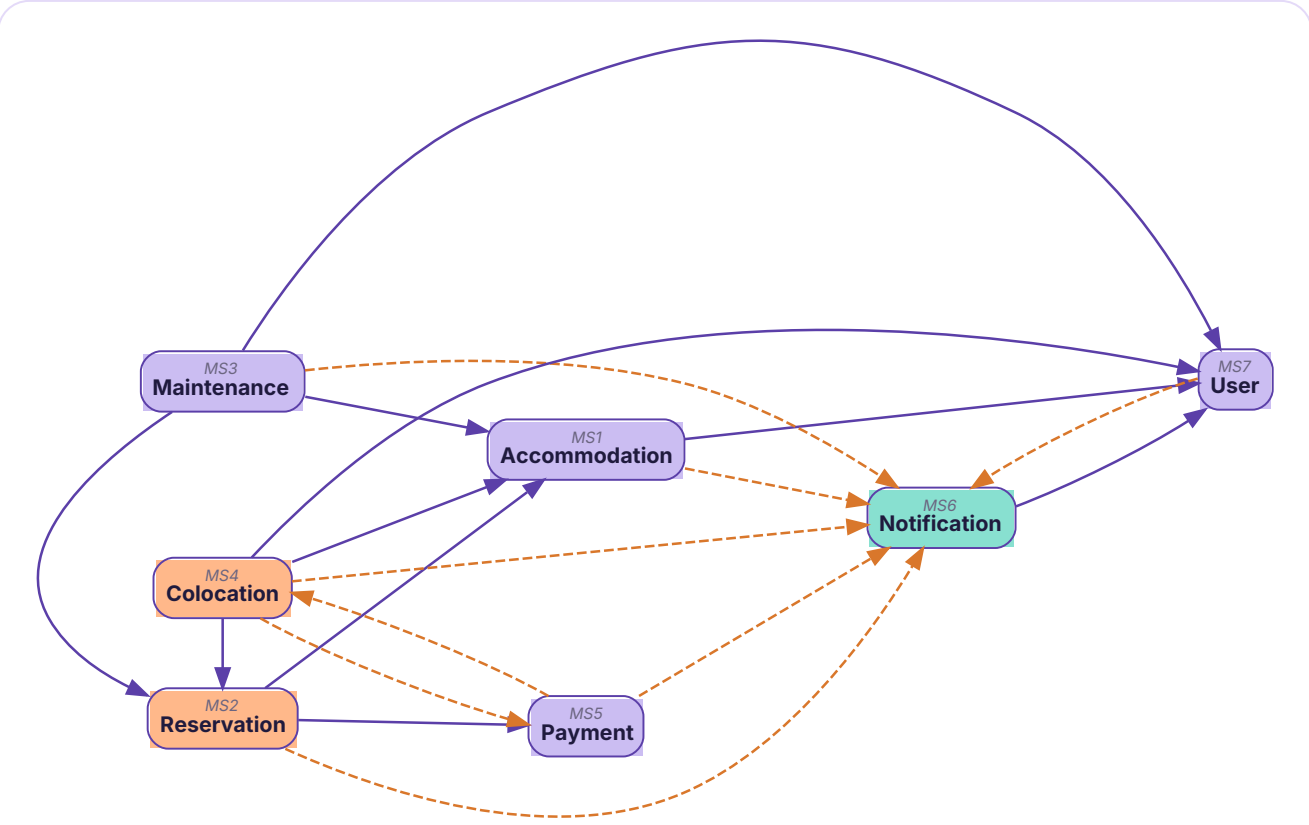


Figure 16 · Carte des interactions entre les sept microservices

synchrone · REST / OpenFeign asynchrone · événements RabbitMQ

Matrice des interactions : la ligne appelle (S) ou publie vers (A) la colonne.

	Accommodation	Reservation	Maintenance	Colocation	Payment	Notification	User
Accommodation	·					A	S
Reservation	S	·			S	A	

Maintenance	S	S	·		A	S
Colocation	S	S		·	A	A	S
Payment				A	·	A	
Notification						·	S
User						A	·

Liste complète des interactions

De	Vers	Mode	Objet de l'échange
Reservation	**Accommodation**	Synchrone	Vérifier la chambre (état, capacité, prix) `GET /api/chambres/{id}/disponibilite`
Reservation	**Payment**	Synchrone	Demander un paiement ou un remboursement `POST /api/paiements` · `POST /api/paiements/{id}/remboursements`
Colocation	**Accommodation**	Synchrone	Informations de la maison d'hôtes, capacité `GET /api/hebergements/{id}`
Colocation	**User**	Synchrone	Identité, profils, rôles des participants `GET /api/users/{id}/profil`
Colocation	**Reservation**	Synchrone	Réservations de groupe, statut `POST /api/reservations`
Maintenance	**Accommodation**	Synchrone	Vérifier la chambre et mettre à jour son état `GET /api/chambres/{id}` · `PATCH /api/chambres/{id}/statut`
Maintenance	**Reservation**	Synchrone	Vérifier l'absence de réservation sur la période `GET /api/reservations/hebergement/{id}/periode`
Maintenance	**User**	Synchrone	Vérifier le technicien (rôle, statut) `GET /api/users/{id}`
Accommodation	**User**	Synchrone	Vérifier le propriétaire de l'hébergement `GET /api/users/{id}`
Notification	**User**	Synchrone	Récupérer coordonnées et préférences du destinataire `GET /api/users/{id}`
Reservation	**Notification**	Asynchrone	Informer après création, confirmation ou annulation `reservation.created` · `reservation.confirmed` · `reservation.cancelled`
Payment	**Notification**	Asynchrone	Informer du résultat d'un paiement ou d'un remboursement `payment.succeeded` · `payment.failed` · `payment.refunded`
Maintenance	**Notification**	Asynchrone	Informer propriétaire et technicien `intervention.created` · `intervention.assigned` · `intervention.closed`

De	Vers	Mode	Objet de l'échange
Colocation	**Notification**	Asynchrone	Invitations, confirmations, mises à jour de groupe `sharedstay.created · member.requested · member.accepted · vote.created`
Accommodation	**Notification**	Asynchrone	Informer d'un changement d'état de chambre `chambre.status.changed`
User	**Notification**	Asynchrone	Message de bienvenue après inscription `user.registered`
Colocation	**Payment**	Asynchrone	Paiement collectif d'un séjour partagé `colocation.payment.requested`
Payment	**Colocation**	Asynchrone	Résultat du paiement collectif `payment.succeeded · payment.failed`

5.3 Interactions du service Accommodation (MS1)

Accommodation est interrogé par Reservation, Colocation et Maintenance. Il vérifie le propriétaire auprès de User et signale les changements d'état de chambre à Notification.

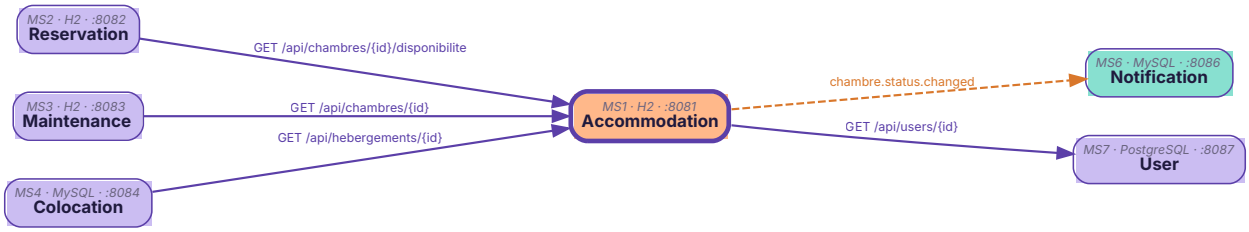


Figure 17 · Interactions du microservice Accommodation

Sens	Service	Mode	Échange
→ appelle	**User** (MS7)	Synchrone	Vérifier le propriétaire de l'hébergement `GET /api/users/{id}`
→ publie	**Notification** (MS6)	Asynchrone	Informer d'un changement d'état de chambre `chambre.status.changed`
← est appelé par	**Reservation** (MS2)	Synchrone	Vérifier la chambre (état, capacité, prix) `GET /api/chambres/{id}/disponibilite`
← est appelé par	**Colocation** (MS4)	Synchrone	Informations de la maison d'hôtes, capacité `GET /api/hebergements/{id}`
← est appelé par	**Maintenance** (MS3)	Synchrone	Vérifier la chambre et mettre à jour son état `GET /api/chambres/{id} · PATCH /api/chambres/{id}/statut`

5.4 Interactions du service Reservation (MS2)

Reservation coordonne trois services (Accommodation, Payment et Notification) et répond aux demandes de Colocation et de Maintenance.

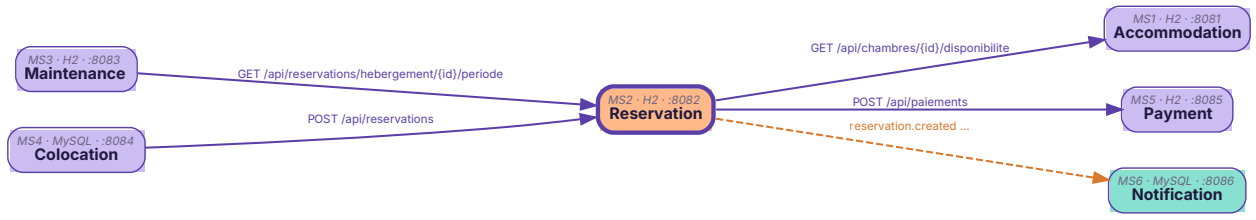


Figure 18 · Interactions du microservice Reservation

Sens	Service	Mode	Échange
→ appelle	**Accommodation** (MS1)	Synchrone	Vérifier la chambre (état, capacité, prix) `GET /api/chambres/{id}/disponibilite`
→ appelle	**Payment** (MS5)	Synchrone	Demander un paiement ou un remboursement `POST /api/paiements · POST /api/paiements/{id}/remboursements`
→ publie	**Notification** (MS6)	Asynchrone	Informer après création, confirmation ou annulation `reservation.created · reservation.confirmed · reservation.cancelled`
← est appelé par	**Colocation** (MS4)	Synchrone	Réservations de groupe, statut `POST /api/reservations`
← est appelé par	**Maintenance** (MS3)	Synchrone	Vérifier l'absence de réservation sur la période `GET /api/reservations/hebergement/{id}/periode`

5.5 Interactions du service Maintenance (MS3)

Maintenance s'appuie sur Accommodation (état des chambres), Reservation (période occupée) et User (technicien), puis informe Notification.

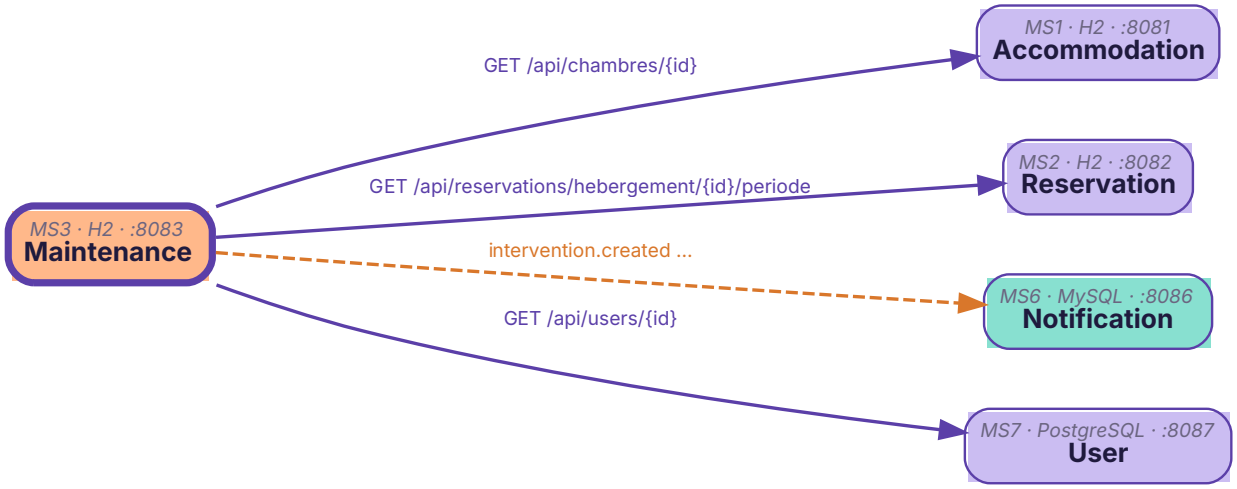


Figure 19 · Interactions du microservice Maintenance

Sens	Service	Mode	Échange
→ appelle	**Accommodation** (MS1)	Synchrone	Vérifier la chambre et mettre à jour son état `GET /api/chambres/{id}` · `PATCH /api/chambres/{id}/statut`
→ appelle	**Reservation** (MS2)	Synchrone	Vérifier l'absence de réservation sur la période `GET /api/reservations/hebergement/{id}/periode`
→ appelle	**User** (MS7)	Synchrone	Vérifier le technicien (rôle, statut) `GET /api/users/{id}`
→ publie	**Notification** (MS6)	Asynchrone	Informer propriétaire et technicien `intervention.created` · `intervention.assigned` · `intervention.closed`

5.6 Interactions du service Colocation (MS4)

Colocation est le service le plus connecté : il appelle Accommodation, User et Reservation en synchrone, et échange avec Payment et Notification par événements.

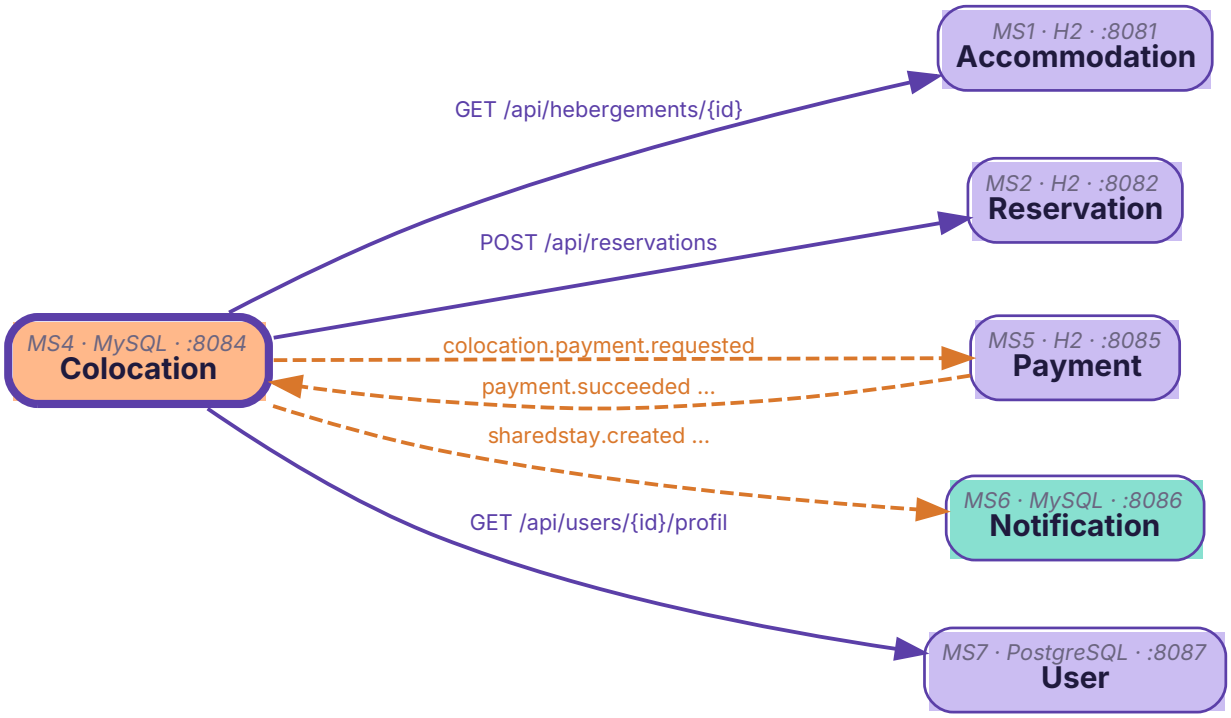


Figure 20 · Interactions du microservice Colocation

Sens	Service	Mode	Échange
→ appelle	**Accommodation** (MS1)	Synchrone	Informations de la maison d'hôtes, capacité `GET /api/hebergements/{id}`
→ appelle	**User** (MS7)	Synchrone	Identité, profils, rôles des participants `GET /api/users/{id}/profil`
→ appelle	**Reservation** (MS2)	Synchrone	Réservations de groupe, statut `POST /api/reservations`
→ publie	**Notification** (MS6)	Asynchrone	Invitations, confirmations, mises à jour de groupe `sharedstay.created` · `member.requested` · `member.accepted` · `vote.created`
→ publie	**Payment** (MS5)	Asynchrone	Paiement collectif d'un séjour partagé `colocation.payment.requested`
← consomme	**Payment** (MS5)	Asynchrone	Résultat du paiement collectif `payment.succeeded` · `payment.failed`

5.7 Interactions du service Payment (MS5)

Payment reçoit les appels de Reservation et les événements de Colocation, puis publie ses résultats vers Notification et Colocation. Il n'appelle aucun service en synchrone.

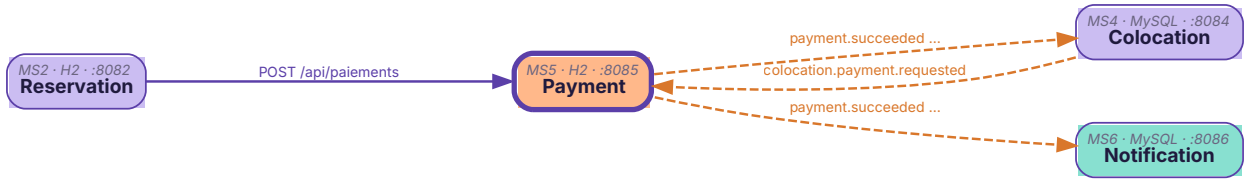


Figure 21 · Interactions du microservice Payment

Sens	Service	Mode	Échange
→ publie	**Notification** (MS6)	Asynchrone	Informer du résultat d'un paiement ou d'un remboursement `payment.succeeded` · `payment.failed` · `payment.refunded`
→ publie	**Colocation** (MS4)	Asynchrone	Résultat du paiement collectif `payment.succeeded` · `payment.failed`
← est appelé par	**Reservation** (MS2)	Synchrone	Demander un paiement ou un remboursement `POST /api/paiements` · `POST /api/paiements/{id}/remboursements`
← consomme	**Colocation** (MS4)	Asynchrone	Paiement collectif d'un séjour partagé `colocation.payment.requested`

5.8 Interactions du service Notification (MS6)

Notification consomme les événements de six services et interroge User pour obtenir les coordonnées du destinataire. Il n'émet aucun événement : l'envoi ne bloque jamais l'appelant.

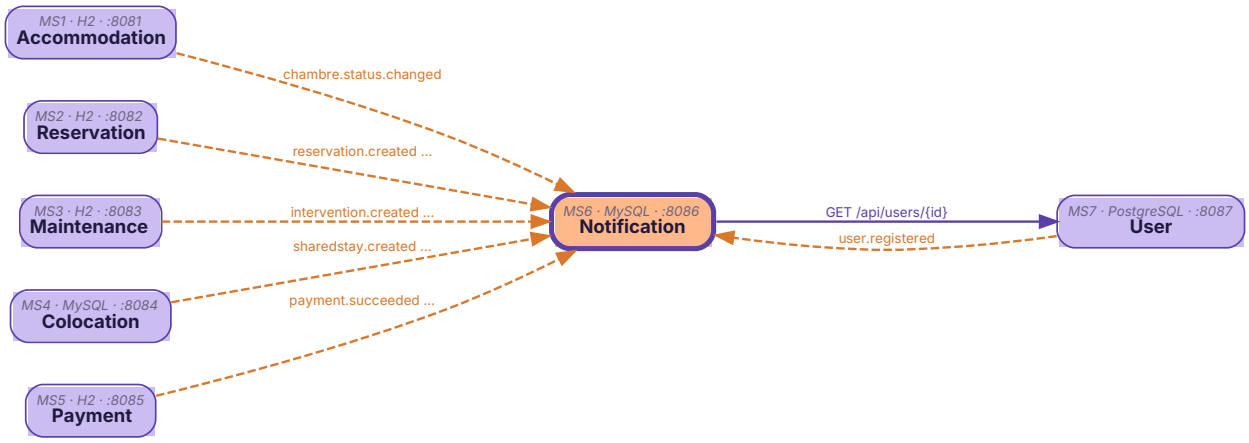


Figure 22 · Interactions du microservice Notification

Sens	Service	Mode	Échange
→ appelle	**User** (MS7)	Synchrone	Récupérer coordonnées et préférences du destinataire `GET /api/users/{id}`
← consomme	**Reservation** (MS2)	Asynchrone	Informer après création, confirmation ou annulation `reservation.created` · `reservation.confirmed` · `reservation.cancelled`
← consomme	**Payment** (MS5)	Asynchrone	Informer du résultat d'un paiement ou d'un remboursement `payment.succeeded` · `payment.failed` · `payment.refunded`
← consomme	**Maintenance** (MS3)	Asynchrone	Informer propriétaire et technicien `intervention.created` · `intervention.assigned` · `intervention.closed`
← consomme	**Colocation** (MS4)	Asynchrone	Invitations, confirmations, mises à jour de groupe `sharedstay.created` · `member.requested` · `member.accepted` · `vote.created`
← consomme	**Accommodation** (MS1)	Asynchrone	Informer d'un changement d'état de chambre `chambre.status.changed`
← consomme	**User** (MS7)	Asynchrone	Message de bienvenue après inscription `user.registered`

5.9 Interactions du service User (MS7)

User est le service le plus sollicité : Accommodation, Maintenance, Colocation et Notification l'interrogent. Il se synchronise avec Keycloak et publie user.registered.

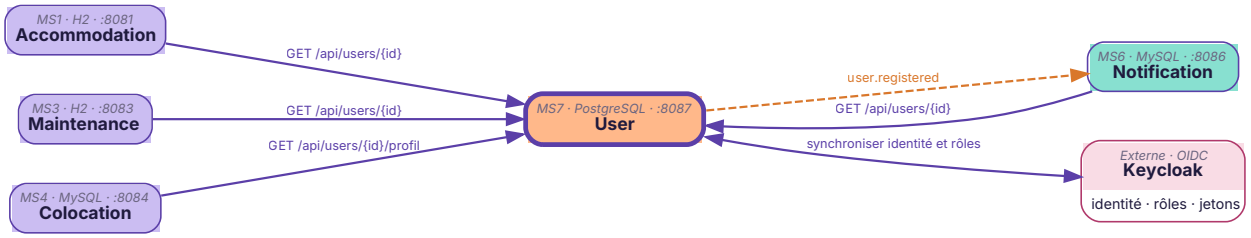


Figure 23 · Interactions du microservice User

Sens	Service	Mode	Échange
→ publie	**Notification** (MS6)	Asynchrone	Message de bienvenue après inscription `user.registered`
← est appelé par	**Colocation** (MS4)	Synchrone	Identité, profils, rôles des participants `GET /api/users/{id}/profil`
← est appelé par	**Maintenance** (MS3)	Synchrone	Vérifier le technicien (rôle, statut) `GET /api/users/{id}`
← est appelé par	**Accommodation** (MS1)	Synchrone	Vérifier le propriétaire de l'hébergement `GET /api/users/{id}`
← est appelé par	**Notification** (MS6)	Synchrone	Récupérer coordonnées et préférences du destinataire `GET /api/users/{id}`
↔ se synchronise avec	**Keycloak** (externe)	Synchrone	Identité, rôles et jetons `Admin REST API · OIDC`

5.10 Scénario : réserver une chambre

Le parcours complet d'une réservation enchaîne trois étapes, avec une gestion explicite des cas d'échec.

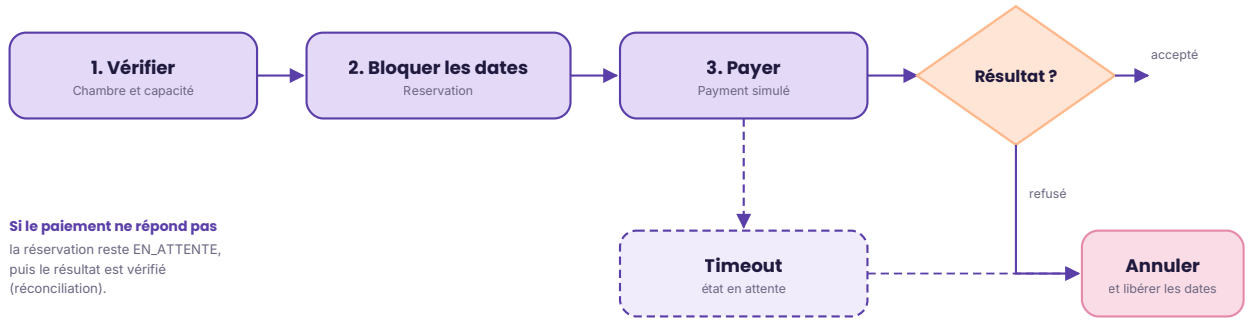


Figure 24 · Parcours métier d'une réservation

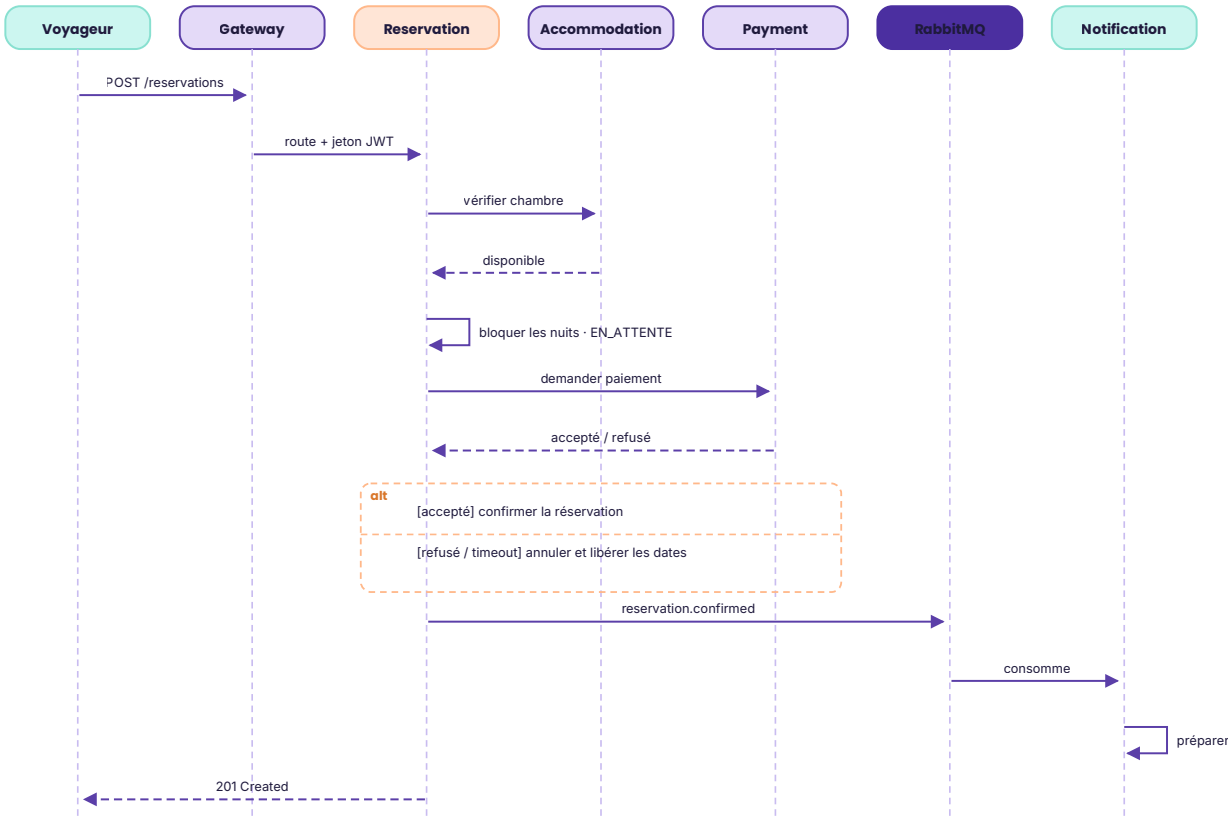


Figure 25 · Diagramme de séquence : création et confirmation d'une réservation

5.11 Scénario : signaler une intervention de maintenance

Avant d'immobiliser une chambre, Maintenance vérifie auprès d'Accommodation que la chambre existe et auprès de Reservation qu'aucune réservation confirmée n'est concernée, puis met à jour l'état de la chambre et informe les acteurs par événement.

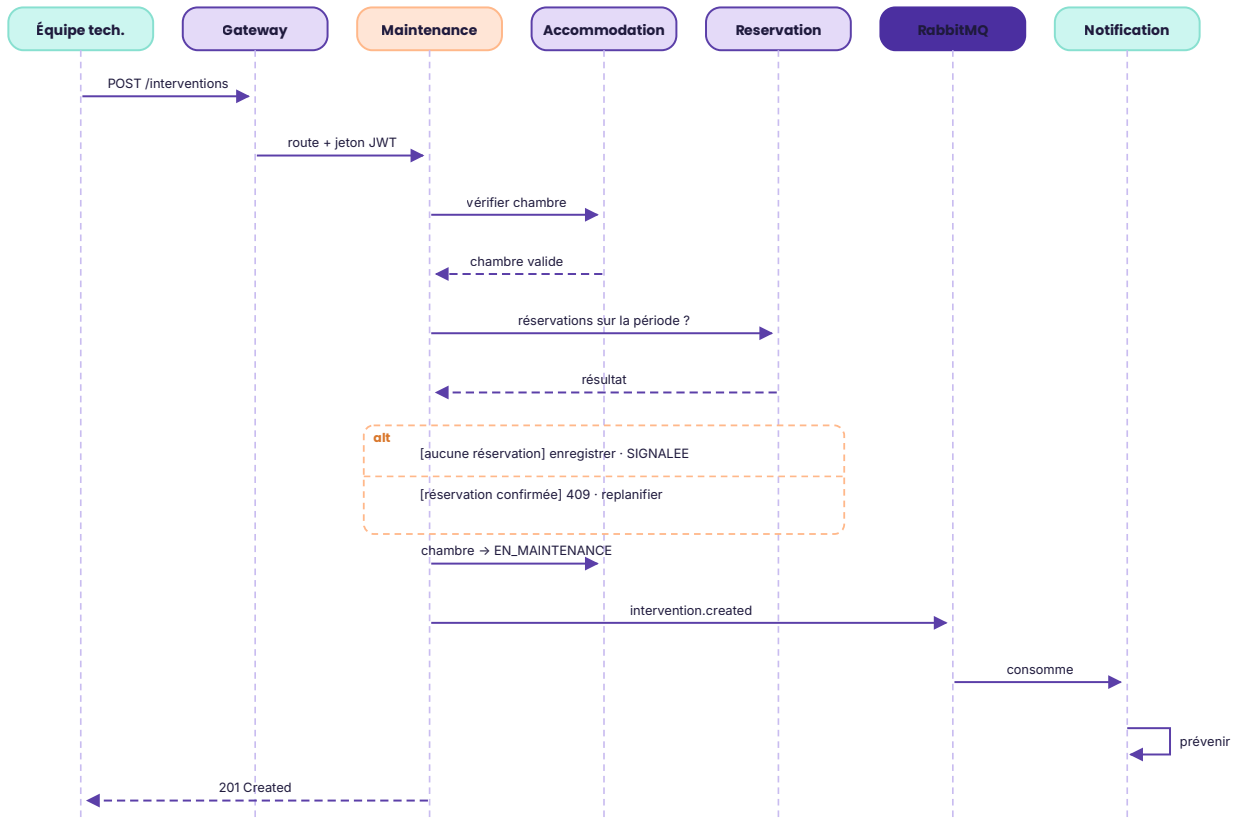


Figure 26 · Diagramme de séquence : signalement d'une intervention

5.12 Scénario : rejoindre un séjour partagé

La candidature est vérifiée auprès de User, puis traitée par l'organisateur ; le paiement collectif passe par des événements entre Colocation et Payment.

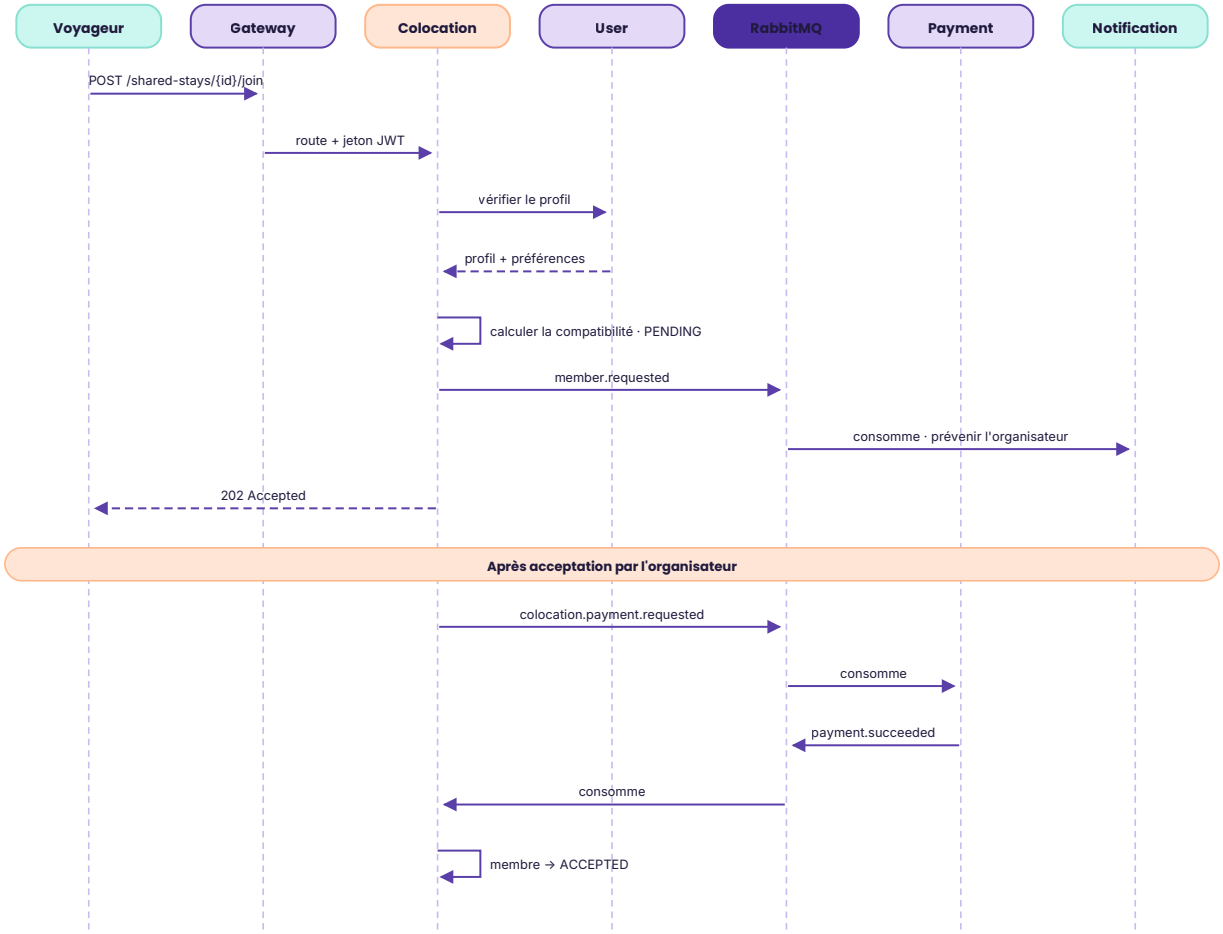


Figure 27 · Diagramme de séquence : candidature et paiement d'un séjour partagé

5.13 Communication asynchrone

Les services métier publient des événements dans RabbitMQ. Le service Notification les consomme pour préparer les messages et alimenter l'historique, sans que l'appelant n'attende. Payment et Colocation échangent aussi par événements pour le paiement collectif.

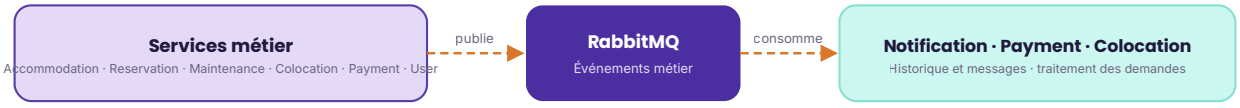


Figure 28 · Publication et consommation des événements

Topologie proposée. Un échange de type *topic* (`stayeasy.events`) reçoit tous les événements ; chaque consommateur possède sa file, reliée par clé de routage, et une file d'échec (dead-letter) pour les reprises. Un identifiant de message unique permet la déduplication côté consommateur.

Catalogue des événements

Événement	Émetteur	Déclencheur	Consommateur
`reservation.created`	**Reservation**	Réservation créée et dates bloquées	Notification
`reservation.confirmed`	**Reservation**	Paiement accepté, réservation confirmée	Notification
`reservation.cancelled`	**Reservation**	Annulation ou paiement refusé	Notification
`payment.succeeded`	**Payment**	Paiement accepté	Notification · Colocation (groupes)
`payment.failed`	**Payment**	Paiement refusé	Notification · Colocation (groupes)
`payment.refunded`	**Payment**	Remboursement effectué	Notification
`intervention.created`	**Maintenance**	Intervention signalée	Notification
`intervention.assigned`	**Maintenance**	Technicien assigné	Notification
`intervention.closed`	**Maintenance**	Intervention clôturée	Notification
`chambre.status.changed`	**Accommodation**	Changement d'état d'une chambre	Notification
`sharedstay.created`	**Colocation**	Séjour partagé créé	Notification
`member.requested`	**Colocation**	Demande pour rejoindre un séjour	Notification
`member.accepted`	**Colocation**	Candidature acceptée	Notification
`vote.created`	**Colocation**	Vote de groupe ouvert	Notification
`colocation.payment.requested`	**Colocation**	Paiement collectif demandé	Payment

Événement	Émetteur	Déclencheur	Consommateur
`user.registered`	**User**	Inscription terminée	Notification

Les événements reservation.created et reservation.confirmed proviennent de la présentation ; les autres sont des propositions de conception.

06 Diagramme d'architecture globale

Vue d'ensemble des couches : accès, sécurité, services métier et communication.

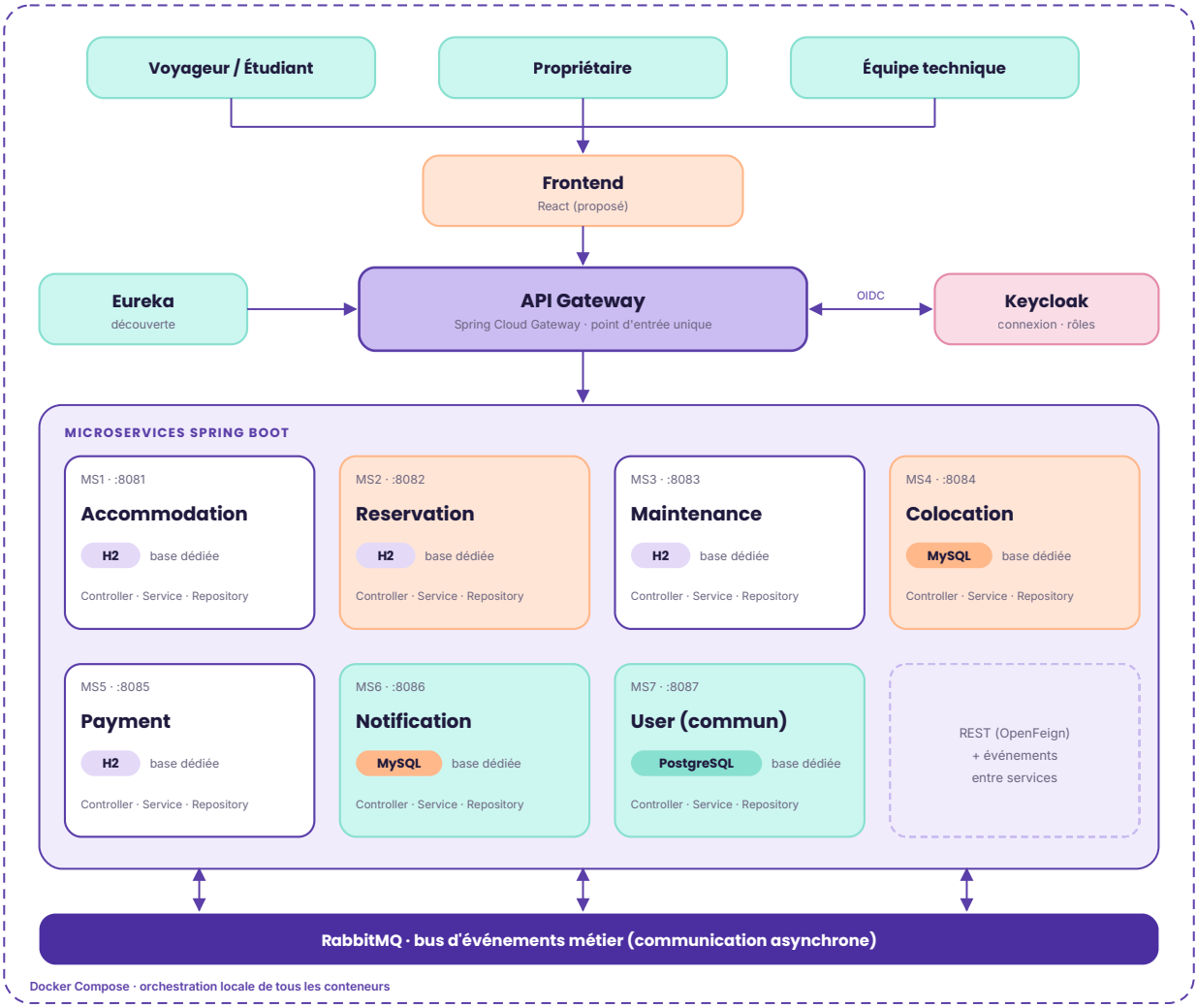


Figure 29 · Architecture globale de StayEasy

6.1 Lecture du diagramme

Couche	Composants	Rôle
Présentation	Frontend React (proposé)	Interface unique pour les trois acteurs.
Accès et sécurité	Spring Cloud Gateway, Keycloak (OIDC)	Point d'entrée unique, authentification et contrôle des rôles.
Découverte	Eureka	Annuaire dynamique des instances de services.
Services métier	Sept microservices Spring Boot	Logique métier, chacun avec sa base dédiée.

Couche	Composants	Rôle
Messagerie	RabbitMQ	Événements métier asynchrones.
Déploiement	Docker Compose	Lancement cohérent de tous les conteneurs.

CODE COULEUR DES BASES

Base	Services
H2	Accommodation, Reservation, Maintenance, Payment
MySQL	Colocation, Notification
PostgreSQL	User

Documentation d'API

Chaque service expose sa documentation Swagger / OpenAPI, ce qui sert de contrat partagé entre les membres de l'équipe.

07 Autres éléments de conception

7.1 Technologies retenues

Backend
Java / Spring Boot
Gateway et Eureka
Swagger / OpenAPI

Données et accès
H2 / MySQL / PostgreSQL
Keycloak : identité
Rôles et autorisations

Intégration
React (proposé)
RabbitMQ (proposé)
Docker Compose

7.2 Conventions d'API et codes de réponse

Toutes les API échangent du JSON, passent par la Gateway et exigent un jeton JWT émis par Keycloak. Les codes ci-dessous sont communs aux sept services ; chaque service précise ses conflits métier dans sa propre section.

Code	Signification
200	Requête réussie
201	Ressource créée
400	Données invalides
401	Jeton absent ou invalide
403	Rôle ou propriétaire non autorisé
404	Ressource introuvable
409	Conflit métier (chambre indisponible, séjour complet, paiement déjà traité…)
500	Erreur interne du serveur
503	Service dépendant indisponible

7.3 Contrats d'API : service Accommodation (MS1)

Méthode	Endpoint	Description
POST	`/api/hebergements`	Créer un hébergement
GET	`/api/hebergements`	Rechercher (ville, type, capacité, prix)
GET	`/api/hebergements/{id}`	Consulter un hébergement
PUT	`/api/hebergements/{id}`	Modifier un hébergement
PATCH	`/api/hebergements/{id}/statut`	Publier ou suspendre
POST	`/api/hebergements/{id}/chambres`	Ajouter une chambre
GET	`/api/hebergements/{id}/chambres`	Lister les chambres
GET	`/api/chambres/{id}/disponibilite`	Vérifier l'état d'une chambre (Reservation)
PATCH	`/api/chambres/{id}/statut`	Mettre à jour l'état (Maintenance)

Codes spécifiques : 409 : chambre EN_MAINTENANCE ou HORS_SERVICE ; 403 : l'utilisateur n'est pas le propriétaire.

EXEMPLE · REQUÊTE

```
POST /api/hebergements
{
  "nom": "Dar Zitouna",
  "ville": "Tabarka",
  "type": "MAISON_HOTES",
  "capaciteMax": 8,
  "prixParNuit": 120.00
}
```

EXEMPLE · RÉPONSE

```
201 CREATED
{
  "id": 12,
  "nom": "Dar Zitouna",
  "statut": "BROUILLON",
  "proprietaireId": 4
}
```

7.4 Contrats d'API : service Reservation (MS2)

Méthode	Endpoint	Description
POST	`/api/reservations`	Créer une réservation
GET	`/api/reservations/{id}`	Consulter une réservation
GET	`/api/reservations/user/{userId}`	Lister les réservations d'un utilisateur
PUT	`/api/reservations/{id}`	Modifier une réservation
PATCH	`/api/reservations/{id}/cancel`	Annuler une réservation
PATCH	`/api/reservations/{id}/confirm`	Confirmer une réservation (après paiement)
GET	`/api/reservations/hebergement/{id}/periode`	Lister les réservations d'une période (Maintenance)

Codes spécifiques : 409 : nuits déjà bloquées ou chambre indisponible ; 503 : service Payment ou Accommodation injoignable.

EXEMPLE · REQUÊTE

```
POST /api/reservations
{
  "hebergementId": 12,
  "dateDebut": "2026-12-15",
  "dateFin": "2026-12-18",
  "userId": 7
}
```

EXEMPLE · RÉPONSE

```
201 CREATED
{
  "id": 301,
  "statut": "EN_ATTENTE",
  "montantTotal": 360.00
}
```

7.5 Contrats d'API : service Maintenance (MS3)

Méthode	Endpoint	Description
POST	`/api/interventions`	Signaler une intervention
GET	`/api/interventions`	Lister (statut, priorité, chambre)
GET	`/api/interventions/{id}`	Consulter une intervention
PUT	`/api/interventions/{id}`	Modifier une intervention
PATCH	`/api/interventions/{id}/assign`	Assigner un technicien
PATCH	`/api/interventions/{id}/status`	Planifier, démarrer ou clôturer
POST	`/api/interventions/{id}/comments`	Ajouter un commentaire
GET	`/api/interventions/{id}/comments`	Lister les commentaires

Codes spécifiques : 409 : réservation confirmée sur la période ou intervention déjà clôturée ; 403 : rôle non autorisé.

EXEMPLE · REQUÊTE

```
POST /api/interventions
{
  "titre": "Fuite lavabo",
  "type": "PLOMBERIE",
  "priorite": "HAUTE",
  "chambreId": 45
}
```

EXEMPLE · RÉPONSE

```
201 CREATED
{
  "id": 88,
  "statut": "SIGNALEE",
  "chambreId": 45
}
```

7.6 Contrats d'API : service Colocation (MS4)

Méthode	Endpoint	Description
POST	`/api/v1/shared-stays`	Créer un séjour partagé
GET	`/api/v1/shared-stays`	Lister les séjours partagés
GET	`/api/v1/shared-stays/{id}`	Récupérer les détails d'un séjour partagé
POST	`/api/v1/shared-stays/{id}/join`	Demander à rejoindre un groupe
GET	`/api/v1/shared-stays/{id}/members`	Récupérer les participants
PATCH	`/api/v1/shared-stays/{id}/members/{memberId}`	Accepter ou refuser une candidature
POST	`/api/v1/matching/calculate`	Calculer la compatibilité
POST	`/api/v1/activities`	Créer une activité de groupe
POST	`/api/v1/votes`	Créer un vote de groupe
POST	`/api/v1/feedback`	Soumettre un retour d'expérience

Codes spécifiques : 409 : séjour complet (maxParticipants atteint) ou candidature déjà existante ; 403 : action réservée à l'organisateur.

EXEMPLE · REQUÊTE

```
POST /api/v1/shared-stays
{
  "title": "Nature Escape",
  "startDate": "2026-12-15",
  "endDate": "2026-12-20",
  "maxParticipants": 5,
  "experienceType": "NATURE",
  "accommodationId": 12
}
```

EXEMPLE · RÉPONSE

```
201 CREATED
{
  "id": 101,
  "title": "Nature Escape",
  "currentParticipants": 1,
  "status": "OPEN"
}
```

7.7 Contrats d'API : service Payment (MS5)

Méthode	Endpoint	Description
POST	`/api/paiements`	Créer un paiement (en-tête Idempotency-Key)
GET	`/api/paiements/{id}`	Consulter un paiement
GET	`/api/paiements/reservation/{reservationId}`	Paiement d'une réservation
GET	`/api/paiements/user/{userId}`	Paiements d'un utilisateur
POST	`/api/paiements/{id}/remboursements`	Rembourser un paiement
GET	`/api/paiements/{id}/remboursements`	Lister les remboursements

Codes spécifiques : 409 : paiement déjà traité (clé d'idempotence) ou remboursement supérieur au montant payé ; 402 : paiement simulé refusé.

EXEMPLE · REQUÊTE

```
POST /api/paiements
Idempotency-Key: 9f1c-77ab
{
  "reservationId": 301,
  "userId": 7,
  "montant": 360.00,
  "devise": "TND",
  "methode": "CARTE_BANCAIRE"
}
```

EXEMPLE · RÉPONSE

```
201 CREATED
{
  "id": 512,
  "statut": "ACCEPTE",
  "reservationId": 301
}
```

7.8 Contrats d'API : service Notification (MS6)

Méthode	Endpoint	Description
GET	`/api/notifications/user/{userId}`	Historique d'un utilisateur
GET	`/api/notifications/{id}`	Consulter une notification
PATCH	`/api/notifications/{id}/read`	Marquer comme lue
GET	`/api/notifications/user/{userId}/unread-count`	Compter les non lues
POST	`/api/notifications`	Envoyer une notification manuelle
GET	`/api/modeles`	Lister les modèles de message
PUT	`/api/modeles/{code}`	Modifier un modèle de message

Codes spécifiques : 404 : notification ou modèle introuvable ; 403 : accès à la notification d'un autre utilisateur.

EXEMPLE · REQUÊTE

```
PATCH /api/notifications/820/read
```

EXEMPLE · RÉPONSE

```
200 OK
{
  "id": 820,
  "type": "RESERVATION",
  "canal": "IN_APP",
  "statut": "LUE"
}
```

7.9 Contrats d'API : service User (MS7)

Méthode	Endpoint	Description
POST	`/api/users`	Créer le profil après connexion
GET	`/api/users/{id}`	Consulter un utilisateur
GET	`/api/users/keycloak/{keycloakId}`	Retrouver un utilisateur via Keycloak
PUT	`/api/users/{id}`	Modifier un utilisateur
GET	`/api/users/{id}/profil`	Consulter le profil (voyageur ou propriétaire)
PUT	`/api/users/{id}/profil-voyageur`	Mettre à jour le profil voyageur
PUT	`/api/users/{id}/profil-proprietaire`	Mettre à jour le profil propriétaire
PATCH	`/api/users/{id}/statut`	Activer ou suspendre un compte

Codes spécifiques : 409 : e-mail ou keycloakId déjà utilisé ; 403 : modification du profil d'un autre utilisateur.

EXEMPLE · REQUÊTE

```
POST /api/users
{
  "keycloakId": "a1b2-c3d4",
  "nom": "Ben Ali",
  "prenom": "Sarra",
  "email": "sarra@mail.tn",
  "role": "VOYAGEUR_ETUDIANT"
}
```

EXEMPLE · RÉPONSE

```
201 CREATED
{
  "id": 7,
  "role": "VOYAGEUR_ETUDIANT",
  "statut": "ACTIF"
}
```

7.10 Sécurité

- Authentification centralisée par Keycloak (protocole OIDC) ; un seul point de connexion pour toute la plateforme.
- Les rôles (voyageur / étudiant, propriétaire, équipe technique) déterminent les opérations autorisées.
- La Gateway est le seul point d'entrée : les services ne sont pas exposés directement.

Droits par rôle et par service

Service	Voyageur / Étudiant	Propriétaire	Équipe technique
Accommodation	Consulter et rechercher	Gérer ses hébergements et chambres	Lire, mettre à jour l'état des chambres
Reservation	Créer, consulter, annuler les siennes	Consulter celles de ses hébergements	—
Maintenance	—	Signaler, consulter les interventions de ses chambres	Gérer les interventions
Colocation	Rejoindre, voter, participer, donner un avis	Créer des séjours, traiter les candidatures	—
Payment	Payer, consulter ses paiements	Consulter les paiements de ses réservations	—
Notification	Consulter ses notifications	Consulter ses notifications	Consulter ses notifications
User	Consulter et modifier son profil	Consulter et modifier son profil	Consulter et modifier son profil

7.11 Risques et mesures prévues dès la conception

#	Risque	Mesure
01	Double réservation	Blocage atomique des nuits.
02	Paiement incertain	Idempotence et réconciliation.
03	Message perdu	Reprise et déduplication.
04	Retard d'intégration	Contrats partagés et priorités.
05	Service dépendant indisponible	Délais d'attente et reprises sur les appels OpenFeign ; réponse 503 explicite.
06	Écart entre services	Références par identifiants, événements idempotents, réconciliation périodique.

7.12 Organisation et répartition du travail

Un service par étudiant

E1 : Accommodation
E2 : Reservation
E3 : Maintenance
E4 : Colocation
E5 : Payment
E6 : Notification

Tout le groupe

Service User
Frontend et intégration
Keycloak, Gateway et Eureka
RabbitMQ et Docker
Documentation et démonstration

Dépôt et méthode de travail

Le dépôt StayEasy est organisé en `Backend/`, `Frontend/` et `Documentation/` (avec `Presentation/` et `Diagrams/`), accompagné d'un `README.md`.

1. **Tâche attribuée** — GitHub Projects
2. **Branche dédiée** — Une fonctionnalité
3. **Pull request** — Relecture collective
4. **Fusion** — Branche main stable

7.13 Planning : de la séance 4 à la séance 10

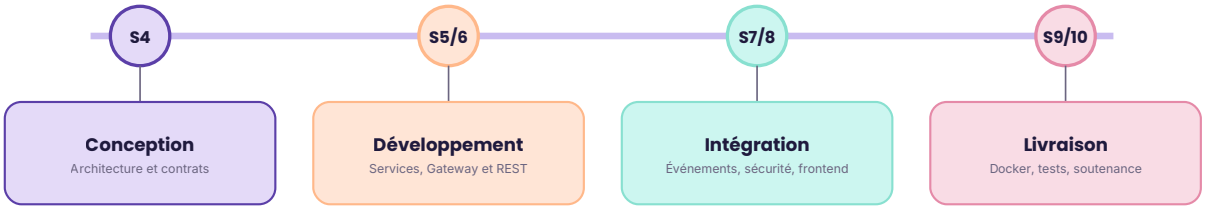


Figure 30 · Planning proposé du projet

08 Conclusion et perspectives

EN UNE PHRASE

Un séjour organisé, des responsabilités bien définies.

2	7	3
parcours : réservation et colocation	services et bases dédiées	acteurs aux rôles distincts

Cette conception répond à la problématique de départ : donner aux voyageurs un parcours simple pour trouver un lieu calme, et aux maisons d'hôtes isolées une visibilité et des outils de gestion. Le découpage en sept microservices à base dédiée, les contrats d'API communs et la communication mixte (REST et événements) offrent une architecture claire, évolutive et répartie équitablement dans l'équipe.

8.1 Perspectives

- Valider la conception : modèles de données, contrats d'API et événements.
- Développer les services, la Gateway et les API REST (séances 5 et 6).
- Intégrer événements, sécurité et frontend (séances 7 et 8).
- Livrer : conteneurisation Docker, tests et soutenance (séances 9 et 10).